Operating System Concepts

- OS Concepts
- Linux commands
- Shell scripts

Learning OS

- step 1: End user
  - Linux commands
- step 2: Administrator
  - Install OS (Linux)
  - Configuration - Users, Networking, Storage, ...
  - Shell scripts
- step 3: Programmer
  - Linux System call programming
- step 4: Designer/Internals
  - UNIX & Linux internals

What is OS?

- Interface between end user and computer hardware.
- Interface between Programs and computer hardware.
- Control program that controls execution of all other programs.
- Resource manager/allocator that manage all hardware resources.
- Bootable CD/DVD = Core OS + Applications + Utilities
- Core OS = Kernel -- Performs all basic functions of OS.

OS Functions

- CPU scheduling

SUNBEAM

- Process Management
- Memory Management
- File & IO Management
- Hardware abstraction
- User interfacing
- Security & Protection
- Networking

Kernel

- The core OS that does basic minimal functionalities of the kernel is referred as "kernel".
- These basic functionalities are
  - CPU scheduling
  - Process Management
  - Memory Management
  - File & IO Management
  - Hardware abstraction layer
- Kernel has four types (as per architecture)
  - Monolithic kernel
  - Micro kernel
  - Modular kernel
  - Hybrid kernel

Monolithic kernel

- All source files are compiled into a single binary kernel image.
- This is kernel is very fast (all components in the same file).
- However if any component fails, whole OS (kernel) crash.
- Modifying any component needs to rebuild whole kernel and redeploy it.
- e.g. Linux (vmlinuz), UNIX.

Microlithic kernel

SUNBEAM

- Kernel is very small including minimal functionalities. So referred as micro-kernel.
- All additional functionalities are implemented as independent processes (called as servers).
- All these servers and kernel communicate with each other by message passing. This reduce performance.
- If any component fails, the rest of system can continue to function. So this kernel is robust.
- Modifying any component needs to rebuild and redeploy only that component.
- e.g. Symbian.

Modular kernel

- Modular kernel is small. Additional functionalities are implemented as dynamically loadable modules (.sys/.ko).
- These modules are loaded into the kernel process at runtime and execute.
- This kernel execute faster, as all modules are part of same kernel process.
- If any component fails, whole OS (kernel) crash.
- Modifying any component needs to rebuild and redeploy only that component.
- e.g. Windows (ntoskrnl.exe)

Hybrid kernel

- This kernel is made by combining source code of multiple kernels.
- e.g. Mac OS X kernel is made up of MACH kernel and BSD UNIX kernel.

Linux kernel

- Linux kernel has static and dynamic components.
- Static components are
  - Scheduler
  - Process management
  - Memory management
  - IO subsystem (core)
  - System calls
- Dynamic components are
  - File systems (like ext3, ext4, xfs)
  - Device drivers

SUNBEAM

- Static components are compiled into the kernel binary image. They are kernel components. The kernel image is /boot/vmlinuz.
- Dynamic components are compiled into kernel objects (*.ko files). They are non-kernel components. They are located in /lib/modules/kernel-version.

Disk/partition naming conventions

```
* Windows:
    * Disks are named as disk0, disk1, ...
    * partitions are named as drives i.e. C:, D:, E:, ...
* Linux:
    * Disks are named as /dev/sda, /dev/sdb, /dev/sdc, etc.
    * Partitions per disk are named as
        * sda partitions: sda1, sda2, sda3, ...
        * sdb partitions: sdb1, ...
```

Path

- It is a unique location of any file in the file system.
- It is represented by character strings with few delimiters (“/”, “\”, “:”)
- Types of path
    - There are two types of paths in linux
    - Absolute path
        - Path which starts with “/” is called as absolute path.
        - E.g. /home/devendra/MyData/Demos/demo01.sh
    - Relative path
        - Path with respect to current directory is called as relative path
        - E.g. MyData/Assignments/assign02.pdf

User interfacing

SUNBEAM

- UI of OS is a program (Shell) that interface between End user and Kernel.
- Shell -- Commmand interpreter
  - End user --> Command --> Shell --> Kernel
- User interfacing (Shell)
  - Graphical User Interface (GUI)
  - Command Line Interface (CLI)

Example shells

- Windows
  - GUI shell: explorer.exe
  - CLI shell: cmd.exe, powershell.exe
- DOS
  - CLI shell: command.com
- Unix/Linux
  - CLI shell: bsh, "bash", ksh, csh, zsh, ...
    - ls /bin/*sh
    - echo $SHELL
  - shell of current user can be changed using "chsh" command.
- GUI shell/standards
  - GNOME: GNU Network Object Model Environment (e.g. Ubuntu, Redhat, CentOS, ...)
  - KDE: Kommon Desktop Environment (e.g. Kubuntu, SuSE, ...)

Linux commands

- cd
  - cd ~ - change working directory to home directory
  - cd - - change working directory to old working directory
  - cd .. - change working directory to parent directory
- ls

SUNBEAM

    - ls - list the contents of present working directory
    - ls path - list the contents of given path

- touch
    - if file does not exist, empty file is created
    - if file exist, timestamp of that file is changed
- pwd
    - pwd - print current working directory
- mkdir
    - mkdir dir - create directory
    - mkdir -p a/b/c - create nested directories
- rmdir
    - rmdir dir - remove empty directory
- rm
    - rm file - remove file
    - rm -r dir - remove directory with contents
    - rm -rf dir - force remove directory
- cat
    - cat file - display file content
    - cat > file - create new file
    - cat >> file - append to file
- Special Characters
    - ~ - home directory

SUNBEAM

- . - current directory
- .. - parent directory
- / - root directory separator
- \ - escape character
- * - matches multiple characters (wildcard)
- ? - matches single character
- > - redirect output (overwrite)
- >> - redirect output (append)
- < - redirect input

SUNBEAM

Program

- Set of instructions given to the computer --> Executable file.
- Program --> Sectioned binary --> "objdump" & "readelf".
  - Exe header --> Magic number, Address of entry-point function, Information about all sections. (objdump -h program.out)
  - Text --> Machine level code (objdump -S program.out)
  - Data --> Global and Static variables (Initialized)
  - BSS --> Global and Static variables (Uninitialized)
  - RoData --> String constants
  - Symbol Table --> Information about the symbols (Name, Size, section, Flags, Address) (objdump -t program.out)
- Program (Executable File) Format
  - Windows -- PE
  - Linux -- ELF
- Program are stored on disk (storage).

Process

- Progam under execution
- Process execute in RAM.
- Process control block contains information about the process (required for the execution of process).
  - Process id
  - Exit status
    - 0 - Indicate successful execution
    - Non-zero - Indicate failure
  - Scheduling information (State, Priority, Sched algorithm, Time, ...)
  - Memory information (Base & Limit, Segment table, or Page table)
  - File information (Open files, Current directory, ...)
  - IPC information (Signals, ...)
  - Execution context (Values of CPU registers)
  - Kernel stack
- PCB is also called as process descriptor (PD), uarea (UNIX), or task_struct (Linux).

SUNBEAM

- In Linux size of task_struct is approx 4KB

Redirection

- for every command input is taken from terminal, output is printed on terminal and error is also printed on terminal
- Standard streams (by defualt for every process, three files are opened)
  - stdin
  - stdout
  - stderr
- There are three types of redirections
  - input redirection
    - input will be taken from file instead of stdin
    - to do input redirection '<' symbol is used
    - command < file
  - output redirection
    - output will be written into file instead of stdout
    - to do output redirection '>' or '>>' symbol is used
    - command > file
      - older content of file will be over written
    - command >> file
      - content will be appneded into file at the end
  - error redirection
    - error will be written into file instead of stderr
    - to do output redirection '2>' or '2>>' symbol is used
    - command 2> file
      - older content of file will be over written
    - command 2>> file
      - content will be appneded into file at the end

SUNBEAM

Pipe

- Using pipe, we can redirect output of any command to the input of any other command.
- Two processes are connected using pipe operator (|).
- Two processes runs simultaneously and are automatically rescheduled as data flows between them.
- If you don't use pipes, you must use several steps to do single task.
- command1 | command2
  - output of command1 will be given as input to command 2
- E.g.
  - who | wc

Linux commands

- cd
  - cd ~ - change working directory to home directory
  - cd - - change working directory to old working directory
  - cd .. - change working directory to parent directory
- ls
  - ls - list the contents of present working directory
  - ls path - list the contents of given path
- touch
  - if file does not exist, empty file is created
  - if file exist, timestamp of that file is changed
- pwd
  - pwd - print current working directory
- mkdir

SUNBEAM

    - mkdir dir - create directory
    - mkdir -p a/b/c - create nested directories
- rmdir
    - rmdir dir - remove empty directory
- rm
    - rm file - remove file
    - rm -r dir - remove directory with contents
    - rm -rf dir - force remove directory
- cat
    - cat file - display file content
    - cat > file - create new file
    - cat >> file - append to file
- head
    - head file - display first 10 lines
    - head -5 file - display first 5 lines
- tail
    - tail file - display last 10 lines
    - tail -4 file - display last 4 lines
- sort
    - sort file - sort the content by alphabetically
    - sort -n file - sort the content by their value
    - sort command do not modify file content
- uniq

SUNBEAM

    - uniq file - display contents uniquely (truncate duplicate)
        - truncate duplicate content if it is consecutive

- rev filepath
    - Print each line reversed.
    - File contents are not modified.
- tac filepath
    - Print all lines in reverse order. The first line printed at last, while last line printed first.

Directory commands

- pwd -- print present working directory (current directory)
- cd -- change directory (syntax> cd dirpath)
- ls - list directory contents (syntax> ls dirpath)
- mkdir -- make directory (syntax> mkdir dirpath)
- rmdir -- remove empty directory (syntax> rmdir dirpath)
- cd
    - cd ~ - change working directory to home directory
    - cd - - change working directory to old working directory
    - cd .. - change working directory to parent directory

File commands

- cat
    - cat > filepath <-- create new file
    - cat filepath <-- display file contents
- rm
    - rm filepath <-- delete given file
    - rm -r dirpath <-- delete dir with all contents
- mv
    - mv filepath destdirpath <-- move given file into given dest directory

SUNBEAM

- mv dirpath destdirpath <-- move given dir into given dest directory
- mv oldname newname <-- rename given file

- cp
  - cp filename newfilename <-- copy file with new name/path.
  - cp filepath destdirpath <-- copy file into given dest dir with same name.
  - cp -r dirpath destdirpath <-- copy file into given dest dir with same name.

SUNBEAM

Types of commands

- Internal commands
  - commands are part of shell program only.
  - No seperate executable is present
  - eg. alias, unalias, cd
- External commands
  - commands are not part of shell program
  - Seperate executable is available in any one of the bin directory
  - eg. ls, cp, mv

Types of Command execution

- Synchronous execution
  - Shell waits for command to be completed.
  - Default execution
- Asynchronous execution
  - Shell doesn't wait for command to be completed.
  - To execute a command asynchronously, use "&" at the end of the command.
  - terminal> gedit &

ls

- ls - list the contents of present working directory
- ls path - list the contents of given path
- ls -l - list the contents in detail format
  - type and permissions
  - Types of files
  - Permissions of files
    - r - read, w - write, x - execute
    - (rwx)user/owner, (rwx)group, (rwx)others
  - link count

Sunbeam

    - user/owner
    - group
    - size
    - timestamp
    - name
- ls -a - list all contents along with hidden
- ls -A - list all contents along with hidden except . and ..
- ls -i - list contents with indoe number
    - inode number is unique number given to every file
- ls -s - list content with size (number of blocks)
- ls -S - list content in descending order of their sizes

Regular Expressions

- Find a pattern in text file(s).
- Regular expressions are patterns used to match character combinations in strings.
- A regular expression pattern is composed of simple characters, or a combination of simple and special characters e.g. /abc/, /ab*c/
- Pattern is given using regex wild-card characters.
    - Basic wild-card characters
        - $ - find at the end of line.
        - ^ - find at the start of line.
        - [ ] - any single char in give range or set of chars
        - [^] - any single char not in give range or set of chars
        - . - any single character
        - zero or more occurrences of previous character
    - Extended wild-card characters
        - ? - zero or one occurrence of previous character
        - one or more occurrences of previous character
        - {n} - n occurrences of previous character
        - {,n} - max n occurrences of previous character
        - {m,} - min m occurrences of previous character
        - {m,n} - min m and max n occurrences of previous character

Sunbeam

        - () - grouping (chars)
        - (|) - find one of the group of characters

grep

- Regex commands
    - grep - GNU Regular Expression Parser - Basic wild-card
    - egrep - Extended Grep - Basic + Extended wild-card
    - fgrep - Fixed Grep - No wild-card
- Command syntax
    - grep "pattern" filepath
    - grep [options] "pattern" filepath
        - -c : count number of occurrences
        - -v : invert the find output
        - -i : case insensitive search
        - -w : search whole words only
        - -R : search recursively in a directory
        - -n : show line number.

VI Editor

- sudo apt-get install vim
- VI editor works in two modes
    - command mode
    - insert mode
- press i - to go into insert mode
- press Esc - to go into command node
- VI editor commands:

Sunbeam

- :w - write/save into file
- :q - quit vi editor
- :wq - save and quit
- :y - to copy current line
- :ny - to copy nth line
- yy - to copy current line
- nyy - copy n lines from current line
- :m,ny - copy fomr mth line to nth line
- :d - to cut current line
- :nd - to cut nth line
- dd - to cut current line
- ndd - cut n lines from current line
- :m,nd - cut fomr mth line to nth line
- press p - to paste copied line on next line of current line
- yw - copy from current position upto next word
- yiw - copy current word
- y$ - copy from cursor position upto end of line
- y^ - copy from cursor position upto start of line
- vim -o - to open multiple files horizontally

Sunbeam

    - vim -O - to open multiple files vertically
    - ctrl + ww - to go into next file
    - /pattern - to search the pattern
    - n - to go on next occurance
    - :s/pattern1/pattern2/ - find and replace only first occurance of current line
    - :s/pattern1/pattern2/g - find and replace all occurances of current line
    - :%s/pattern1/pattern2/g - find and replace all occurances of file
- To do setting in vi editor
    - create file into home directory as below:

```
vim ~/.vimrc
```

- add below content into it

```
set number
set autowriteall
set autoindent
set tabstop=4
set nowrap
```

Sunbeam

sed command

syntax

```
sed [OPTIONS] 'COMMAND' [INPUTFILE...]
```

OPTIONS

- -i : Edit the file in-place (overwrite)
- -n : Suppress automatic printing of lines.
- -e : Allows multiple commands.
- -f : Reads sed commands from a file.
- -r : Use extended regular expressions.

COMMAND

- s/old_word/new_word/ - substitute first occurrence of old_word by new_word
- s/old_word/new_word/n - substitute nth occurrence of old_word by new_word
- s/old_word/new_word/g - substitute all occurrences of old_word by new_word
- s/old_word/new_word/gi - case sensitive substitute all occurrences of old_word by new_word
- s/old_word/new_word/ng - substitute from nth occurrence of old_word by new_word
- n s/old_word/new_word/ - substitute first occurrence of old_word by new_word on nth line
- m,n s/old_word/new_word/ - substitute first occurrence of old_word by new_word from mth to nth line
- m,$ s/old_word/new_word/ - substitute first occurrence of old_word by new_word from mth to last line
- s/old_word/new_word/p - duplicate replaced lines
- nd - delete nth line
- $d - delete last line
- m,nd - delete mth to nth line
- m,$d - delete mth to last line
- /pattern/d - delete pattern matching line

SUNBEAM

- = - print file with line numbers

```
cat > sample.txt
Linux is a powerful operating system.
Unix is the origin of Linux and many Unix systems are still in use.
Linux and Unix are both multiuser systems.
This system is used in servers, and this system is very reliable.
Many developers prefer Linux because Linux is open source.
Unix systems were developed earlier than Linux systems.
This line contains Linux Linux Linux multiple times.
Unix Unix systems are known for stability.
The system administrator manages the Linux system.
EOF
```

```
# replace words start with L
sed 's/\bL[a-zA-Z]*/OS/g' sample.txt

# Replace only whole word "Linux"
sed 's/\bLinux\b/OS/g' sample.txt

# Replace lines starting with "Unix"
sed 's/^Unix/OS/' sample.txt

# Replace lines ending with "system."
sed 's/system\.$/OS./' sample.txt

# Replace multiple spaces with one space
sed 's/  */ /g' sample.txt

# Print lines with either Linux OR Unix
sed -n '/Linux\|Unix/p' sample.txt

# Replace digits
```

SUNBEAM

```
sed 's/[0-9]/X/g' sample.txt

# Insert text before line 3
sed '3i\new text' filename

# Insert text after line 3
sed '3a\new text' filename
```

Regex symbols

- ^ - Start of line
- $ - End of line
- . - Any character
- *- 0 or more
- + - 1 or more
- ? - 0 or 1
- [] - Character class
- \b - Word boundary
- | - OR
- () - Grouping

awk command

- awk is a powerful text-processing command in Linux used to analyze, filter, and manipulate structured data such as logs, CSV files, and command output.
- It works by scanning input line by line and performing actions based on patterns and fields.
- Processes text column-wise using fields (e.g., $1, $2)
- Ideal for parsing logs, reports, and delimited files
- Supports conditions, loops, and built-in functions
- Commonly used in shell scripts and data processing pipelines

syntax

SUNBEAM

```
awk [options] 'pattern {action}' input-file

# [options]: Controls AWK behavior (e.g., -F to set field separator)
# pattern: Specifies which lines to process (condition or regex)
# {action}: Defines what to do with matching lines (usually print)
```

options

- -F : to specify field seperator
- -f : to specify script file
- -v : to define a variable

Built-In Variables In Awk

- $0, $1, $2, ...
    - $0 - represents entire input line
    - $1, $2, ... - represent individual fields (columns).
- NR (Number of Records)
    - Keeps track of the current line number being processed.
- NF (Number of Fields)
    - Stores the total number of fields in the current line.
- FS (Field Separator)
    - Defines the character used to split fields in the input.
    - Default is whitespace
- RS (Record Separator)

SUNBEAM

  - Specifies how input records are separated.
  - Default is a newline character

- OFS (Output Field Separator)
  - Controls the separator used between fields in output.
  - Default is a space and is applied when using commas in print.

- ORS (Output Record Separator)
  - Defines how output lines are separated.
  - Default is a newline, and it is automatically added after each print.

```
# print all lines
awk '{print}' students.csv

# print 1st and 2nd column of each line
awk -F ',' '{print $1, $2}' students.csv

# print msg before processing
awk -v msg="Student Details:" 'BEGIN {print msg}{print}' students.csv

# print students whose roll no is less than 20
awk -v limit=20 '$1 < limit {print $1, $2 }' students.csv

# search lines with a keyword
awk '/pune/ {print}' students.csv

# print all input lines with line numbers
awk '{print NR, $0}' students.csv

# print first and last column of input line
awk -F ',' '{print $1,$NF}' employee.txt

# display lines from 2 to 3
```

SUNBEAM

```
awk 'NR == 2 , NR == 3 {print NR, $0}' students.csv

# to print empty line if any
awk 'NF <= 0 {print NR}' students.csv

# to print number of lines in input file
awk 'END { print NR }' students.csv

# find length of longest line in a file
awk '{ if (length($0) > max) max = length($0) } END { print max }' students.csv
```

SUNBEAM

ls

- ls -a - list all contents along with hidden
- ls -A - list all contents along with hidden except . and ..
- ls -i - list contents with indoe number
    - inode number is unique number given to every file

File Permissions/Mode

- File permission types: r (read), w (write), x (execute)
- Permission levels: u (user), g (group), o (other)
- File mode: u-rwx g-rwx o-rwx
- Example: hello.txt -- user can read/write and execute, group can read and execute, other can only read.
    - hello.txt mode: u=rwx g=r-x o=r--
        - rwx r-x r--
        - 111 101 100 = 754
    - chmod 754 hello.txt

chmod

- chmod permissions filename
- Permissions:
    - +x/r/w - add permission to all
    - -x/r/w - remove permission of all
    - u+x/r/w - add permission to user
    - u-x/r/w - remove permission of user
    - g+x/r/w - add permission to group
    - g-x/r/w - remove permission of group
    - o+x/r/w - add permission to others
    - o-x/r/w - remove permission of others
    - 0774 - apply rwx to user, rwx to group and r to others

SUNBEAM

chown

- sudo chown username filename
- sudo chown :groupname filename
- sudo chown username:groupname filename

Directory permissions

- r (read): Can read directory entries.
  - ls command can list the directory.
  - readdir() can get directory entry.
- w (write): Can add new directory entry, modify existing directory entry, or delete directory entry.
  - new directory entry: when create new sub-directory (e.g. mkdir) or file (e.g. touch, ...).
  - modify existing directory entry: rename file or sub-directory.
  - delete directory entry: delete file or sub-directory (e.g. rm, rmdir, ...)
- x (execute): Can browse the directory
  - cd command (chdir() syscall) will work

Types of Command execution

- Synchronous execution
  - Shell waits for command to be completed.
  - Default execution
- Asynchronous execution
  - Shell doesn't wait for command to be completed.
  - To execute a command asynchronously, use "&" at the end of the command.
  - terminal> gedit &

File Management

File

- File is collection of data/information on storage device.

SUNBEAM

    - File = Contents (Data) + Information (Metadata)
    - The data is stored in zero or more Data blocks (in FS), while metadata is stored in the FCB (in filesystem).
- FCB is called as "inode" on UNIX/Linux. It contains
    - type: UNIX/Linux has 7 types of files
        - -: regular, d: directory, l: symbolic link, p: pipe, s: socket, c: char device, b: block device
    - size: number of bytes
    - links: number of hard links
    - mode (permissions): (u) rwx, (g) rwx, (o) rwx
    - user & group
    - time-stamps: modification, creation, access.
    - info about data blocks
- terminal> ls -l
    - type, mode, links, user, group, size, timestamp, name.
- terminal> stat filepath

File System

- Files are stored on storage device. Arrangement of files in storage device is called as "File System".
- e.g. FAT, NTFS, EXT2/3/4, ReiserFS, XFS, HFS, etc.
- File System logically divide partition into 4 sections.
    - Boot block/Boot sector
        - Contains programs/info required for booting of OS
        - Typically contains bootstrap program and bootloader program
    - Super block/Volume control block
        - Contains information of whole partition.
        - Capacity, Label.
        - terminal> df -h
            - Total number of data blocks/inodes.
            - Number of used/free data blocks/inodes.
            - Information of free data blocks/inodes.

SUNBEAM

    - Inode List/Master file table
        - Inodes (FCB) for each file
    - Data blocks
        - Stores data of the file.
        - Each file have zero or more data blocks.
        - Size of data blocks can be configured while creating file system

- File system is created by the format utility while formatting the partition.
    - Windows: format.exe
    - Linux: mkfs
        - terminal> sudo mkfs -t ext3 /dev/sdb1
        - terminal> sudo mkfs -t vfat /dev/sdb1
        - -t fs_type e.g. ext3, ext4, vfat, ntfs, ...
        - partition e.g. /dev/sdb1

- Disk/partition naming conventions
    - Windows:
        - Disks are named as disk0, disk1, ...
        - partitions are named as drives i.e. C:, D:, E:, ...
    - Linux:
        - Disks are named as /dev/sda, /dev/sdb, /dev/sdc, etc.
        - Partitions per disk are named as
            - sda partitions: sda1, sda2, sda3, ...
            - sdb partitions: sdb1, ...

SUNBEAM

Service Management

Services and Daemons

- While the terms service and daemon are sometimes used interchangeably, there are specific differences between the two
- Daemons are noninteractive programs running on the system. They do not provide direct communication with users and operate in the background.
- Services are interactive, though they too operate in the background. Users may interact with services. Services are the purpose behind server computers.
- Note that services are usually implemented as daemons and also contain an interactive component. Hence, services such as SSH will have a related daemon, such as sshd.
- Service and daemon management is an essential part of the sysadmin role
- Here are a few critical services that users rely on throughout the business day
    - Web service: Makes web pages available across the network and relies on httpd
    - SSH service: Allows secure remote administration and relies on sshd
    - NFS service: Allows remote access to exported directories and relies on several daemons, including nfsd and mountd
    - NTP service: Synchronizes time among network servers and relies on ntpd

System Initialization

- System initialization is the process that begins when the kernel first loads
- It involves the loading of the operating system and its various components, including the boot process
- System initialization is carried out by an init daemon in Linux—the "parent of all processes."
- The init daemon refers to a configuration file and initiates the processes listed in it
- This prepares the system to run the required software
- The init daemon runs continuously until the system is powered off, and programs on the system will not run without it-
- On Linux, there are two main methods that initialize a system: systemdand SysVinit. Which method is active on your Linux system will affect how you manage services on that system
- Most modern Linux distributions use systemd to manage initialization deprecated
- However, some distributions, especially older ones, rely on a different initialization mechanism named System V

systemd

SUNBEAM

- The systemd software suite provides an init method for initializing a system
- It also provides tools for managing services on the system that derive from the init daemon
- The systemd suite is now the dominant init method in modern Linux distributions, and was designed as a replacement for other methods like SysVinit
- The systemd suite offers several improvements over older methods
- For example, it supports parallelization (starting programs at the same time for quicker boot) and reduces shell overhead
- In systemd, Control Groups (cgroups) are used to track processes instead of process IDs (PIDs), which provides better isolation and categorization for processes

systemctl command

- One of the most frequent tasks for administrators is managing services and daemons
- Management includes starting, stopping, and checking the status of these applications
- It's critical to remember that Linux services and daemons acquire their settings from configuration files that are read when the system or the applications start
- Any changes to these settings occur by editing the files, which must then be reread to discover the new configurations
- In addition, administrators may choose whether a given service starts when the system boots--
- Managing Services with systemctl
    - start: Start the service or daemon (not persistent)
    - stop: Stop the service or daemon (not persistent)
    - restart: Restart the service or daemon
    - enable: Set the service or daemon to start at boot (persistent)
    - disable: Set the service or daemon not to start at boot (persistent)
    - status: Display the current status of the service or daemon

systemd unit files

- Under the systemd initialization process, the term unit refers to a resource that systemd knows how to manage via daemons and configuration files
- This system is much more flexible than earlier initialization methods
- Unit files define how systemd manages the unit
- There are many systemd unit file types, with .service, .timer, .mount, and .target as the primary examples
- Stored in:
    - /etc/systemd/system: custom

SUNBEAM

    - /usr/lib/systemd/system: default
- Commands
    - Use systemctl –t helpto get list of service units
    - Use systemctl list-unit-files for listing unit files
    - Use systemctl list-units --type=service for filtering units with type service

Unit file types

- .service: Background services
- .socket: IPC/network sockets
- .target: Group of units (like runlevels)
- .mount: Filesystem mounts
- .automount: On-demand mounts
- .timer: Scheduled jobs
- .path: File-based triggers
- .device: Hardware devices
- .swap: Swap partitions

System Targets

- A target in systemdis a group of units (services, mounts, etc.) that represents a system state
- A mode of operation (CLI mode, GUI mode, rescue mode)
- A replacement for traditional runlevels
    - 0 - poweroff.target (Shutdown the system)
    - 1 - rescue.target (Single user mode)
    - 2 - multi-user.target (Multi-user without networking)
    - 3 - multi-user.target (CLI (without GUI) with networking)
    - 4 - Reserved
    - 5 - graphical.target (With GUI mode)
    - 6 - reboot.target (Reboot the system)
- Check current target: sudo systemctlget-default
- Set a target: sudo systemctl set-default

SUNBEAM

Package Management

Managing Software

- Maintaining software on Linux systems is usually accomplished with package managers
- Package managers deploy, update, remove, and report on applications
- However, due to the open-source nature of the Linux environment, software applications are often compiled from source code-something that is not common with other operating systems
- Because there are many different Linux distributions available, there are many package managers
- As a general rule, most distributions either originated with Red Hat Linux (and therefore use the Red Hat package managers) or Debian Linux (and therefore use the Debian package managers)

Software Packages

- Linux software is often distributed in packages that contain everything needed to run the software—executables, configuration files, supporting resources, and documentation
- Therefore, the process of installing, maintaining, and removing software is referred to as package management, and the tools required to maintain software are package managers
- One critical component of package managers is the ability to report software information
    - Package managers track software installation information in a database that can be queried
    - Such queries might include a list of all installed software, a list of software versions, or specific information on a single software package
    - The ability to query the package manager's installation database provides sysadmins with essential information on the system and its applications
- A second critical part of package management is the source from which package managers pull the software
    - A storage location for software packages is referred to as a repository---
    - Sysadmins control what repositories are available to package managers
- Repositories can be located in a few key areas:
    - An internal server containing only authorized packages
    - A distribution vendor, such as Red Hat or Ubuntu
    - A public package repository

Red Hat vs Debian

SUNBEAM

- The Red Hat and Debian package managers handle the software lifecycle in much the same way
- Each approach has a method for installing, updating, removing, and reporting software on the system
- Package managers track versions, the location of installed files, and place documentation files in standard locations
- Each package manager has its own related commands, and those commands cover the three major parts of the software lifecycle (installation, maintenance, and removal)
- Red Hat Package Managers
    - Red Hat Package Manager (RPM)
    - Yellow Dog Update Manager (YUM)
    - Dandified YUM (DNF) package manager
- Debian Package Managers
    - dpkg package manager
    - Advanced Packaging Tool (APT)
- Suse Zypp package manager
    - SUSE Linux uses its own package manager for software maintenance: Zypp
    - The ZYpp package manager is the basis for the GUI tool YaST (Yet another Setup Tool) and the command-line zypper utility

Software Dependencies

- Package managers make software maintenance much more straightforward
- One challenging part of handling Linux software, however, is dependencies
- Some software packages rely on other packages to function
- If the software an application depends on is not installed, it will either fail entirely or not work as intended
- Early package managers required administrators to manually provide the dependent applications before installing the primary package
- Modern package managers such as YUM, DNF, and APT install supporting applications automatically, ensuring that a single installation command provides the user with everything needed to run the program
- Periodically, package dependency issues still arise, so recognize the problem (failed dependency) and the solution (install the packages required for the primary application to run)

Red Hat Package Manager

RPM package

SUNBEAM

- An RPM package is essentially an archive file that contains:
  - Compiled software binaries-executable
  - Configuration files
  - Metadata (name, version, description, dependencies)
  - Installation scripts (pre-install, post-install, pre-uninstall, post-uninstall)-
  - File manifests (list of all files included)
  - Digital signature (GPG)
- Features of rpm packages
  - Dependency management: RPM tracks what packages require what libraries. If you try to install a package without its dependencies, RPM will show an error
  - Verification: RPM can verify that installed files haven't been tampered with
  - Digital signature: RPM packages are GPG-signed to ensure authenticity and integrity
  - Database tracking: RPM maintains a local database (/var/lib/rpm/) that tracks:
    - All installed packages
    - All files belonging to each package
    - Package metadata

Red Hat Package Manager

RPM

- The Red Hat Package Manager (RPM) is the oldest of the package managers
- It provides good basic functionality with a simple syntax and a lot of flexibility---
- It maintains a software database for package queries---
- One of its main drawbacks is package management, which is not as robust as YUM and DNF

YUM

- Yellow Dog Update Manager (YUM) relies on .rpm packages and also tracks software in an RPM database
- Like RPM, YUM alerts users to software dependencies---
- However, it also offers the ability to automatically install dependencies so that a single YUM command installs the specified software and all required software for it to function

SUNBEAM

DNF

- Both RPM and YUM are superseded by Dandified Yum (DNF), which includes all the same functionality as both with additional plugins and features
- Typically, any Linux system that uses .rpm software packages recognizes RPM, YUM, and DNF commands

RPM Commands

- The use of Red Hat package managers is primarily focused on installing, updating, upgrading, and uninstalling software packages
- These packages have a .rpm file extension
- RPM files are pre-compiled and usually include all supporting documentation and configuration files
- The idea is that a single package contains the entire application
- Commands
  - rpm -ivh
  - rpm -Uvh
  - rpm -evh

Basic package management

- yum install : install a package
- yum update : update a package
- yum update –security: apply only security-related updates
- yum upgrade: upgrade packages considering obsolete
- yum downgrade : downgrade a package
- yum remove : remove or uninstall a package
- yum autoremove: remove unnecessary dependencies
- yum reinstall : reinstall current version of package
- yum history: show yum history

Query and Search

- yum list installed: list all installed packages
- yum list available: list all packages available to install-

SUNBEAM

- yum list all: list installed and available packages
- yum list updates: list available updates
- yum info : display detailed information about a package
- yum search : search for packages by name or description
- yum provides <file_path>: find which package owns a specific file
- yum deplist : list the package's dependencies
- yum check-update: check updates without installing them

Repositories

- When the rpm, yum, or dnf commands are issued to install a package, the commands must know where to look for the software
- If a package has been downloaded to a specific file, the package manager can be directed to that file with the absolute path
- However, locally downloaded and stored software is not recommended
- Version control is difficult, as is enforcing standard applications
- Therefore, administrators should define centralized repositories of packages and configure the YUM and DNF package managers to use those repositories when installing or updating software
- Types
    - Local repositories: These repositories are stored on the system's local storage drive. Installation is easy, but version control is difficult because of the decentralized nature of local repositories. If you manage 20 Linux servers, each would have its own repository to be maintained
    - Centralized internal repositories: These repositories are stored on one or more systems within the internal LAN and managed by the Linux administrator. This centralized approach makes version control much simpler. If you manage 20 Linux servers, one could host the repository and the other 19 could download their packages from it
    - Vendor repositories: These repositories are maintained on the Internet, often by the distribution vendor. Version control is very difficult because the vendor decides what package versions are made available

Repository File structure

- The .repo files are stored in the /etc/yum.repos.d/ directory-
- Some of the components of the .repo file include:
    - [repo-name] —The repository name.--
    - name=Repository Name —The human-friendly name of the repo.
    - baseurl= —The path to the repo. May be a file (file:///) or http:// path.-

SUNBEAM

    - enabled=1 —Enables the repo.
    - gpgcheck=0 —Disables GPG checking
- yum repolist: shows the list of repositories

Debian Package Manager

Debian package

- A Debian package is the standard software distribution format for Debian and its derivatives (Ubuntu, Linux Mint, etc.)
- These packages, which use the .deb file extension, contain compiled software along with metadata that enables the package management system to install, upgrade, and remove software efficiently
- The Debian package management system, called dpkg, forms the foundation of software management across the entire Debian ecosystem
- Two types of debian packages
    - Binary Packages
        - Binary packages (.deb files) contain pre-compiled executables, libraries, configuration files, documentation, and metadata
        - These are what users typically install on their systems. The binary package includes everything needed to run the software on a specific computer architecture
    - Source Packages
        - Source packages consist of three components :
            - .dsc file (Debian Source Control) - Contains metadata about the source package
            - .orig.tar.gz - The original upstream source code
            - .debian.tar.xz - Debian-specific modifications and packaging files
        - Source packages allow users to rebuild the software from scratch or inspect how the Debian maintainers customized the original software

Debian package manager

- The Debian package management system is one of the most mature and robust software management frameworks in the Linux ecosystem
- It uses .deb packages as its core format and employs a multi-layered approach to handle software installation, removal, updating, and dependency resolution
- This system forms the foundation not only for Debian itself but also for its derivatives including Ubuntu, Linux Mint, and many others

Core Components

SUNBEAM

- dpkg - The Low-Level Package Manager
  - dpkg (Debian Package) is the fundamental, low-level tool that directly manipulates .deb package files
  - It installs, removes, and queries packages but does not automatically handle dependencies
  - Think of it as the engine that performs the actual installation work
  - Key characteristics of dpkg:
    - Works directly with .deb files
    - Maintains a local database in /var/lib/dpkg/
    - Provides powerful querying capabilities
    - Requires manual dependency resolution
    - Essential for low-level operations and troubleshooting
- APT - The Advanced Package Tool
  - APT is the high-level package management system that builds on top of dpkg. It automates the entire process by:
    - Automatically resolving and installing dependencies
    - Managing remote software repositories
    - Handling package updates and upgrades
    - Providing secure package verification via GPG signatures

Basic package managemen

- apt update: Refresh package lists from repositories
- apt list --upgradable: Show packages that can be upgraded
- apt install package: Install a package with dependencies
- apt remove package: Remove package (keep configs)
- apt purge package: Remove package and all config files
- apt autoremove: Remove unnecessary dependencies
- apt upgrade: Upgrade all installed packages
- apt full-upgrade: Upgrade with intelligent dependency handling

Query and searching packages

- apt search keyword: Search for packages by keyword
- apt show package: Display detailed package information

SUNBEAM

- apt list --installed: List all installed packages

Repositories

- Debian repositories are configured in two locations:
- /etc/apt/sources.list - Main configuration file
- /etc/apt/sources.list.d/*.list - Directory for additional sources

SUNBEAM



Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

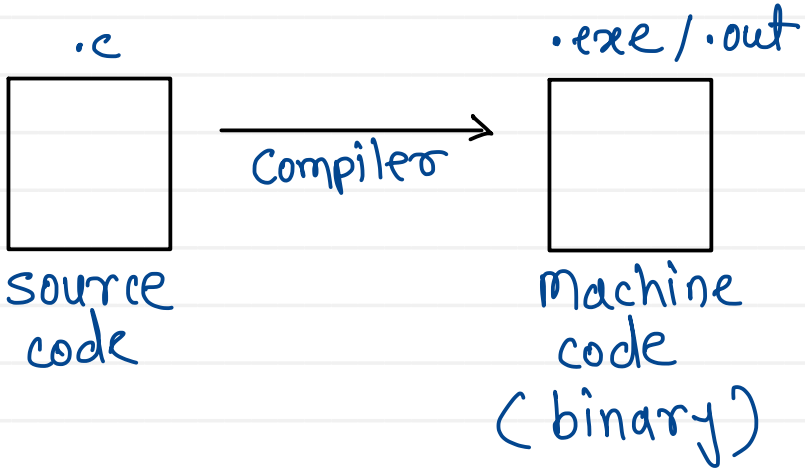
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

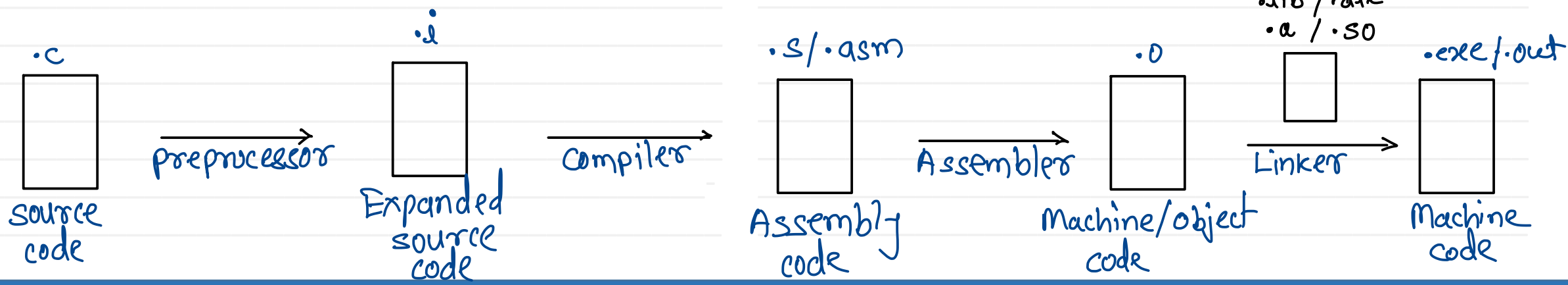
Program compilation steps

GCC - GNU C Compiler
GNU - GNU Not UNIX

Process : Program in execution
Program : set of instructions for machine (CPU)



Toolchain : (gcc)
- set of tools which work on source code one by one to convert it into machine code.
1. Preprocessor (cpp)
2. Compiler (cc)
3. Assembler (as)
4. Linker (ld)
5. Debugger (gdb)
6. Utilities (make, objdump)



Program



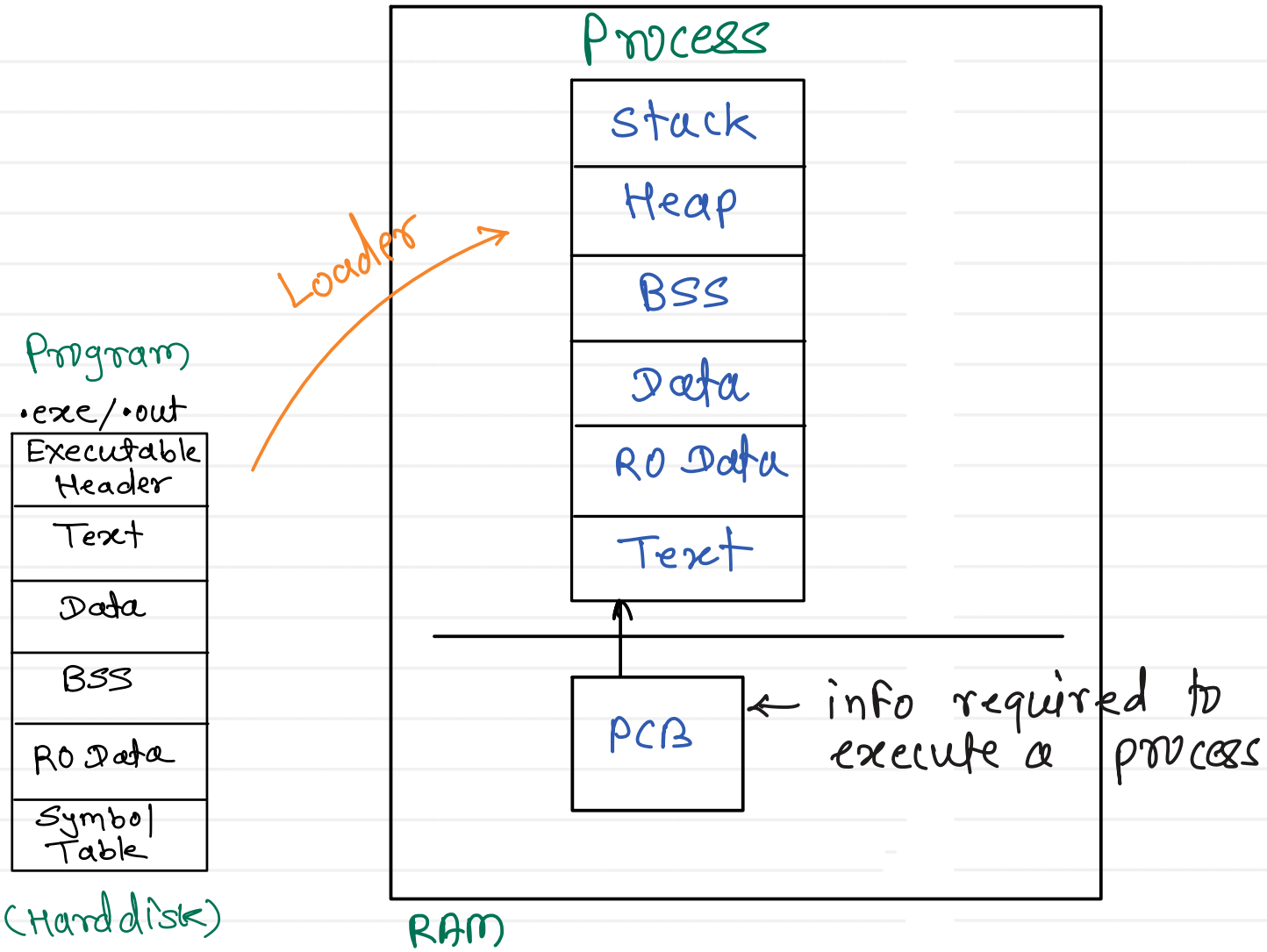
- info required to execute a program
- magic number (identify to file format) (2 or 4 bytes)

Windows - .exe → Portable Executable → MZ
Linux - .out → Executable Linking format → 2. ELF

- type of program (CLI / GUI / Library)
- info about remaining sections (start, end, size)
- addr of entry point function (main)
- instructions of a program in machine code format
- initialized static & global variables
- uninitialized static & global variables
- read only data (string constants)
- info about symbols (function & variables)

variables — type, size, addr, section, default value
functions - return type, name/addr, No. of args, type of args

Process

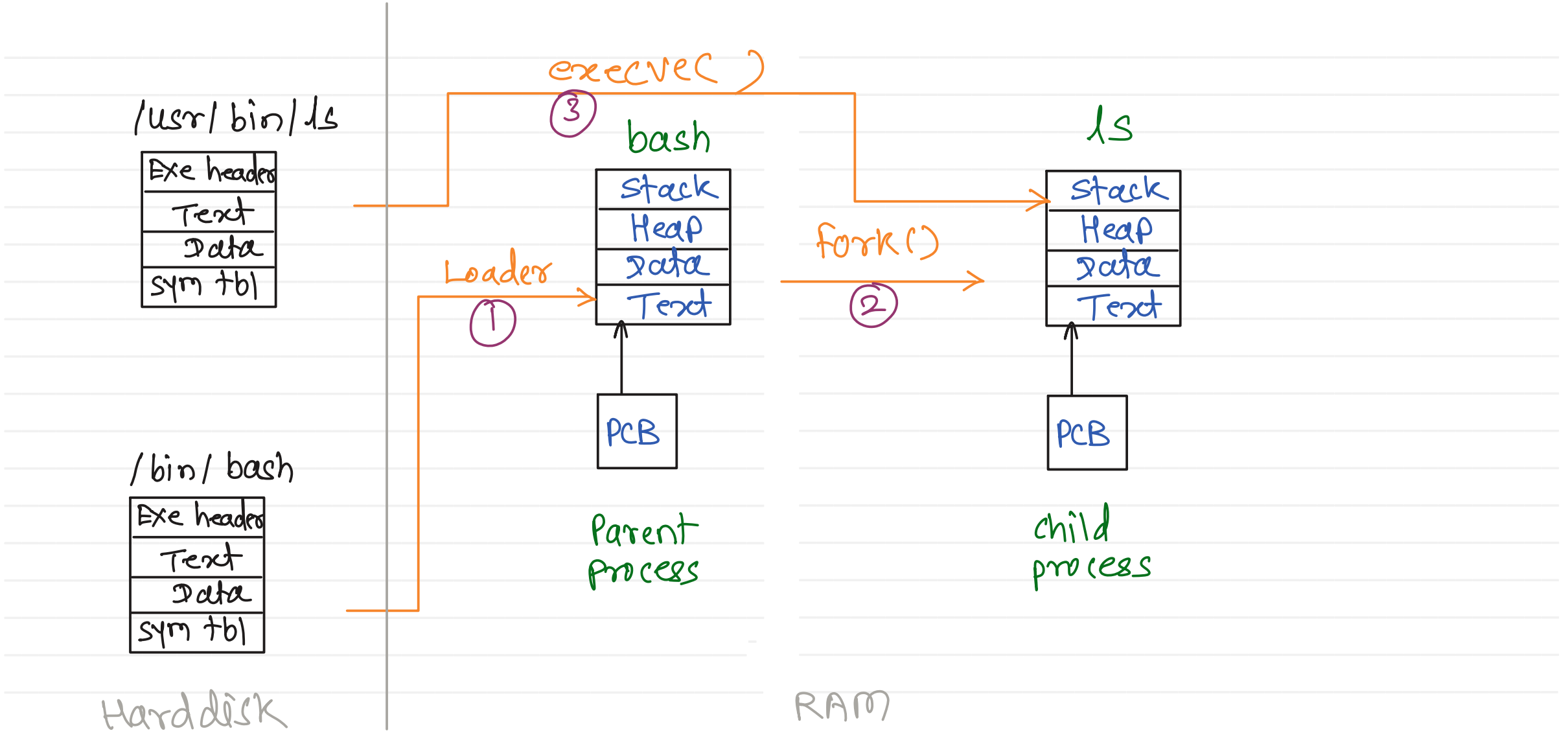


To read executables :
readelf / objdump
xxd

PCB - process descriptor
↓
struct task_struct
(sched.h)

- pid , ppid
- exit status
- CPU sched info
- Mem info
- IPC info — message queue
  - pipe
  - signals
  - socket
  - shared memory
- File info
- ....

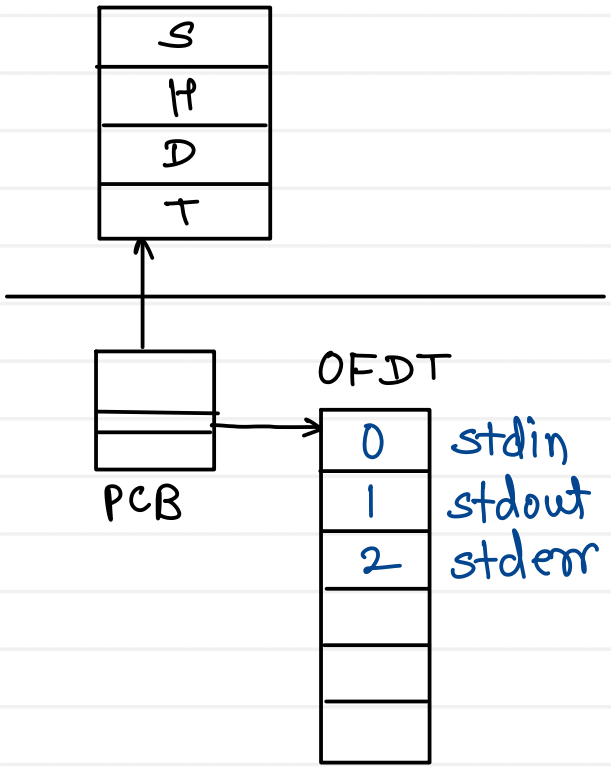
Shell internals - how commands executes?



Redirection

- by default, every program
  takes input from standard input (stdin)
  writes output on standard output (stdout)
  writes error on standard error (stderr)

e.g. cat

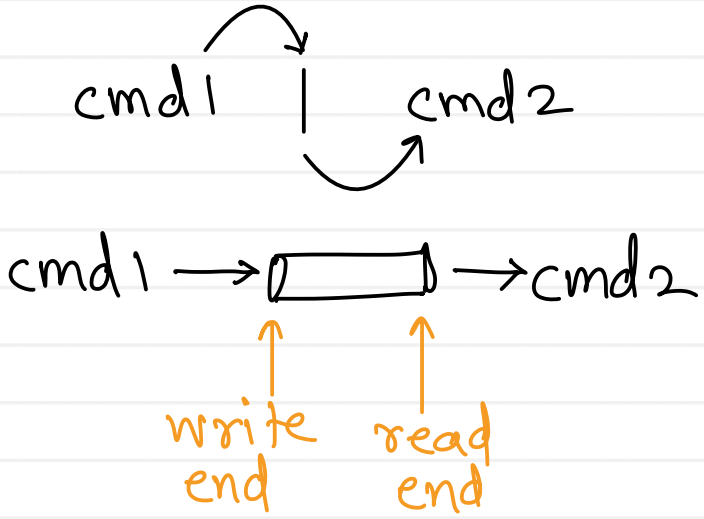


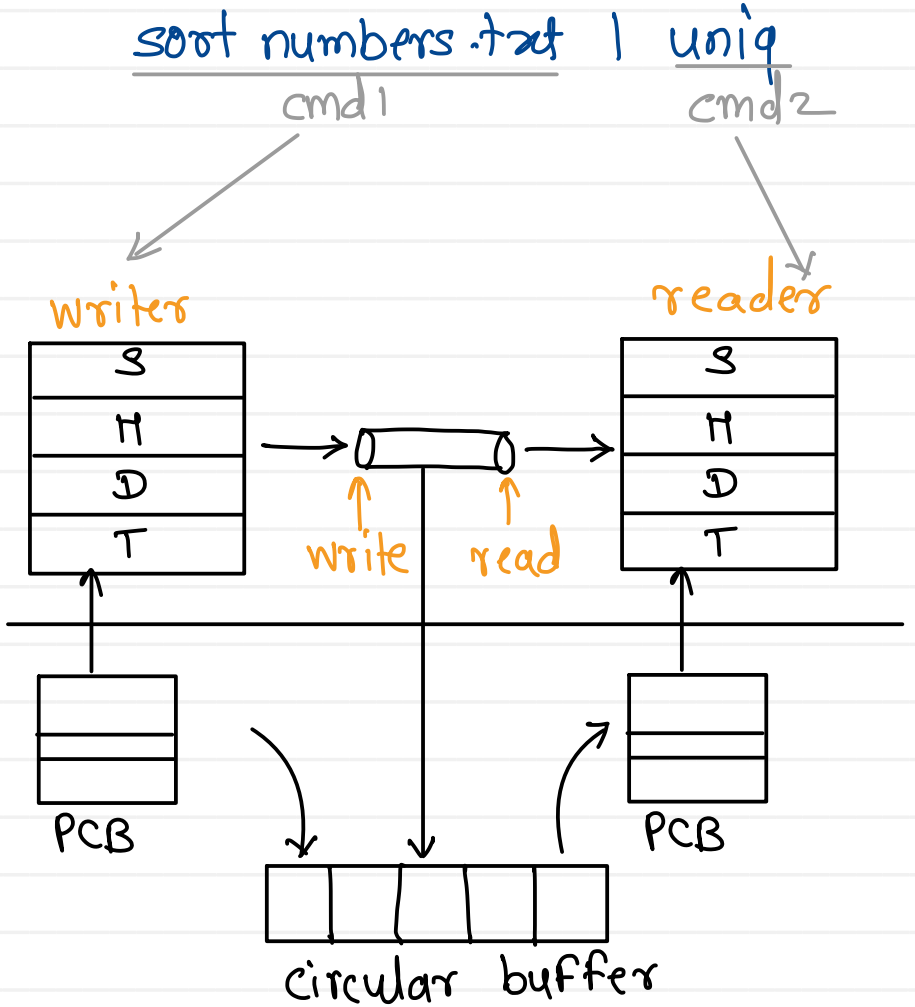
- we can change this default behaviour by doing 'redirection'

- There are three types of redirections
  1. Output redirection (>) (>>)
     instead of writing output on stdout it is written into file
  2. input redirection (<)
     instead of taking input from stdin, it is taken from file
  3. error redirection (2>) (2>>)
     instead of writing error on stderr it is written into file

Pipe

- pipe can be used to do nesting of multiple commands
- output of first command is given to input of second command.
- pipe is a unidirectional IPC mechnism







Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

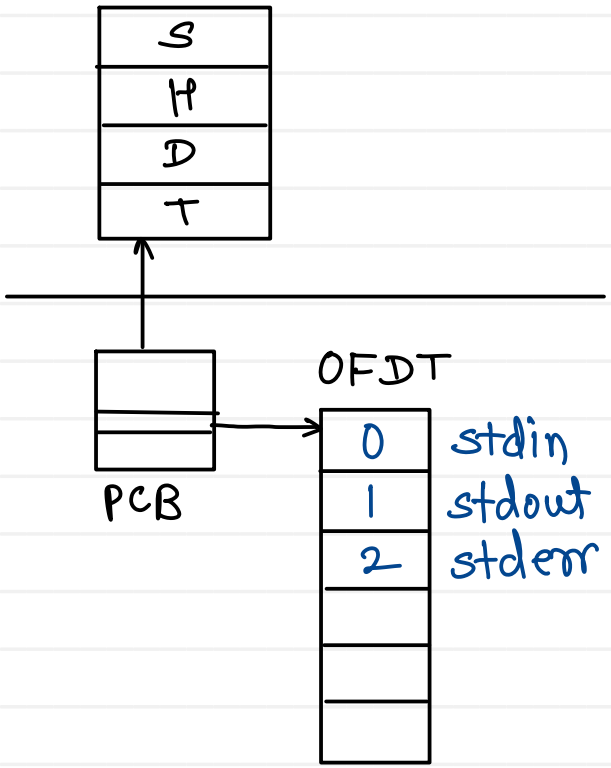
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Redirection

- by default, every program
  takes input from standard input (stdin)
  writes output on standard output (stdout)
  writes error on standard error (stderr)

e.g. cat

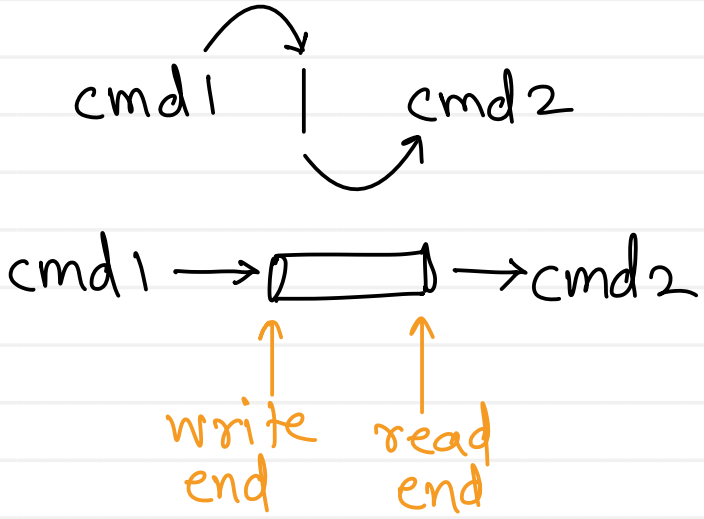


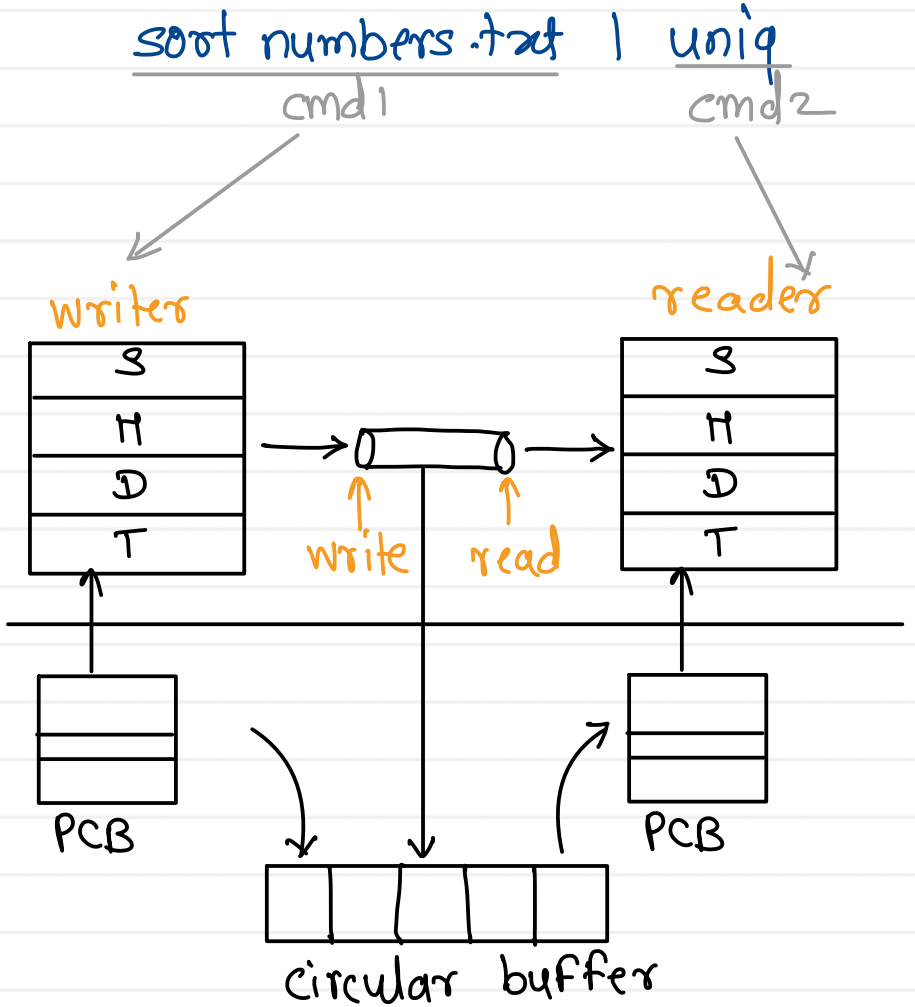
- we can change this default behaviour by doing 'redirection'

- There are three types of redirections
  1. Output redirection (>) (>>)
     instead of writing output on stdout it is written into file
  2. input redirection (<)
     instead of taking input from stdin, it is taken from file
  3. error redirection (2>) (2>>)
     instead of writing error on stderr it is written into file

Pipe

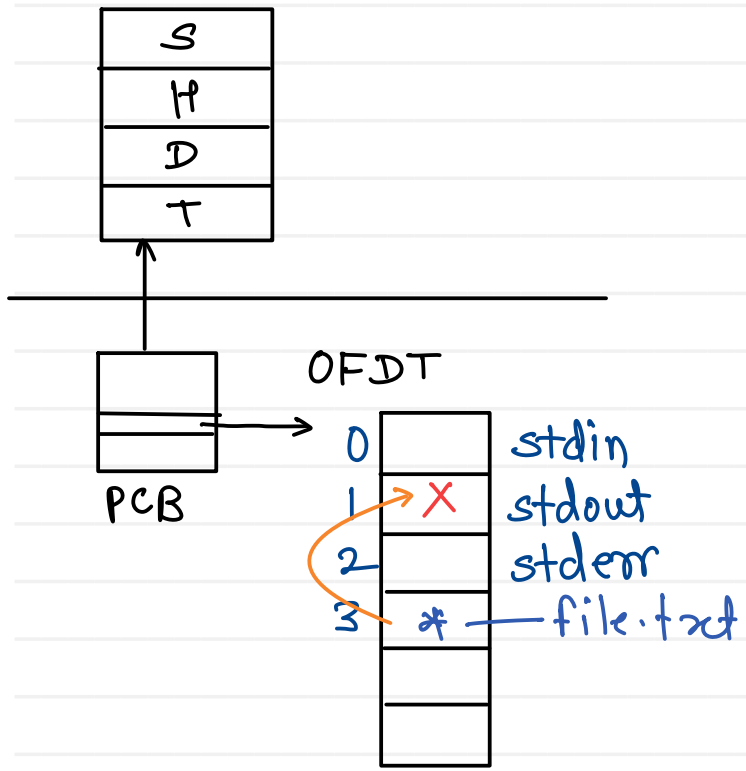
- pipe can be used to do nesting of multiple commands
- output of first command is given to input of second command.
- pipe is a unidirectional IPC mechnism



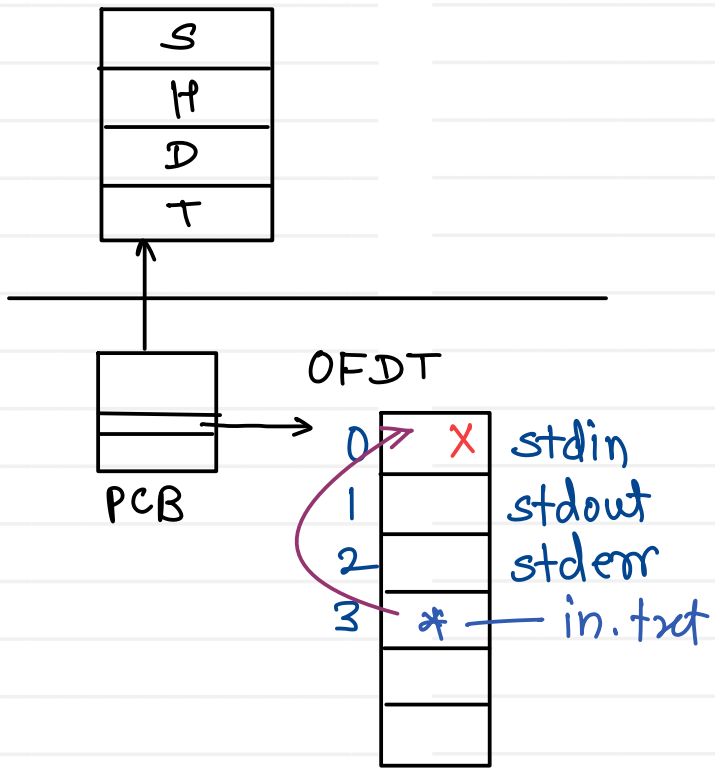


Redirection

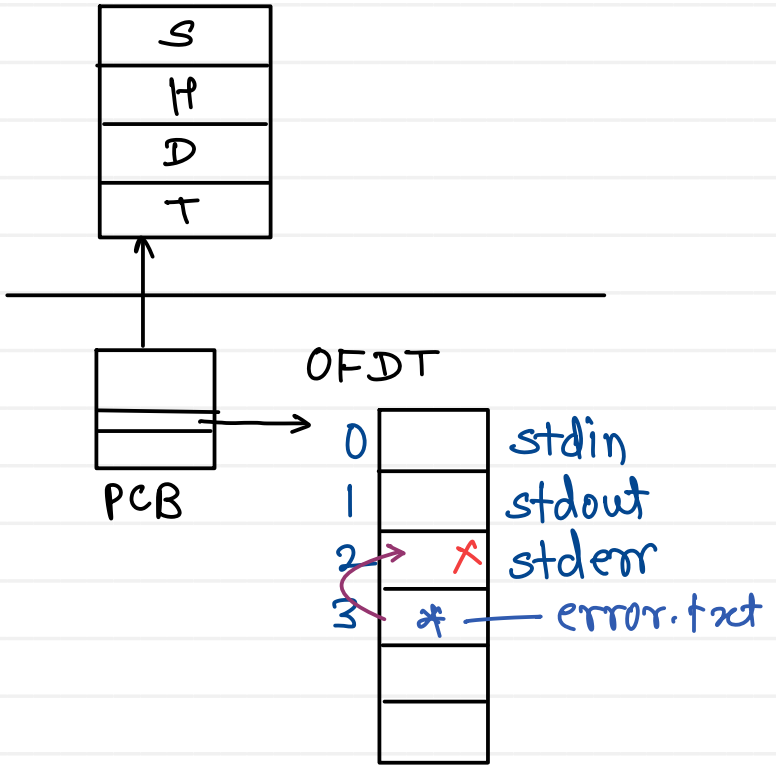
cat > file.txt



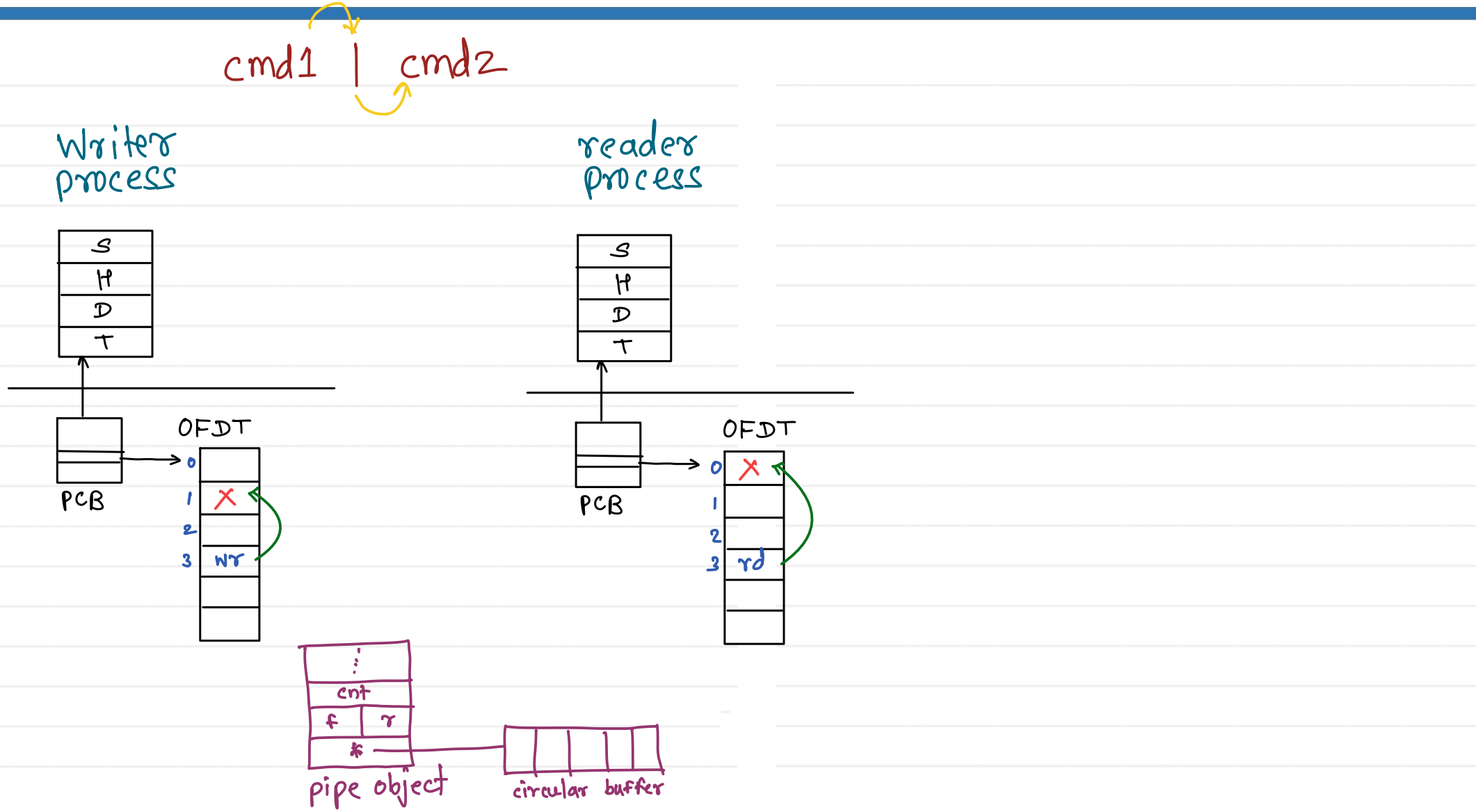
bc < in.txt



ls -W 2> error.txt



Pipe


cmd1 | cmd2
Writer process
reader process
S
H
D
T
PCB
OFDT
0
1
2
3
WR
rd
cnt
f
r
*
pipe object
circular buffer

File

File : collection of data

File = data + metadata

(Actual content of a file)

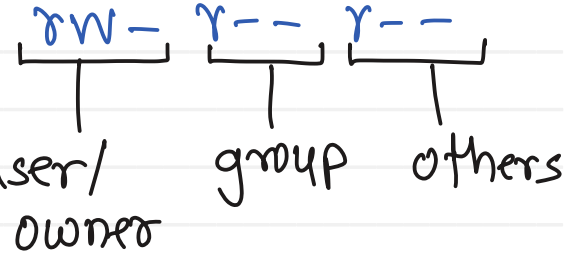
(information about file)

1. name / path
2. size
3. timestamps
   (Created / Accessed / Modified)
4. owner / group
5. permissions / mode / Access
   r - read, w - write, x - execute
   (owner/user, group, others)
6. type of file
7. No. of links

Types of file :

1. Regular file (-)
2. Directory file (d)
3. link file (l)
4. pipe file (p)
5. socket file (s)
6. char special file (c)
7. block special file (b)

permissions:



commands

```
# get user's info
# if user does not exist, the command will return an error
> id sunbeam

# create a new user
> sudo useradd user1

# create a new user with different location of home directory
> sudo useradd --home-dir /tmp/user1 user1

# create a new user with expiration date
> sudo useradd -e 2023-10-25 user1

# create a new user with GECOS field value
> sudo useradd --comment user1 user1


# delete a user without home directory
> sudo userdel user1

# delete a user with home directory
> sudo userdel -r user1


# change the password of current user
> passwd

# change the password of a user
> sudo passwd user1




# add a user
> sudo useradd <username>

# add a group
> sudo groupadd <group name>

# change user's primary group to another group
> sudo usermod -g <group name> <user name>

# append user to another group or change user's supplementary group
> sudo usermod -aG <group name> <user name>

# remove a user from a supplementary group
> sudo usermod -rG <group name> <user name>
```

exercise 1

make sure that both bob and alis do not exist on the machine
create a user named bob with password bob123
set the password interactively (getting user's input)
create another user name alis with password alis123
set the password non-interactively (setting password using command)
> echo "alis:alis123" | sudo chpasswd
check if you can login as bob and alis
get both user's ids
delete both the users along with their home directories

```bash

# exercise 2

# change the max password days to 30
# change the warning password days to 15
# change the min password length to 6

# exercise 3

# create a new user named bob with password bob123
# make sure that when bob logs in, he should see a file named welcome.txt in his home directory

# exercise 4

# create 4 users dev1, dev2, tester1, tester2

# create 3 groups dev, tester, employee

# change the membership for dev1 and dev2 to dev

# change the membership for tester1 and tester2 to tester

# add dev1, dev2, tester1 and tester2 to employee group
```

```
# set password to all the users
```



Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

File - types, permissions, groups

type | No. of hard links | size (in bytes) | name

`-rw-rw-r--  1  sunbeam  sunbeam  86  Sep 23 04:33  move.txt`

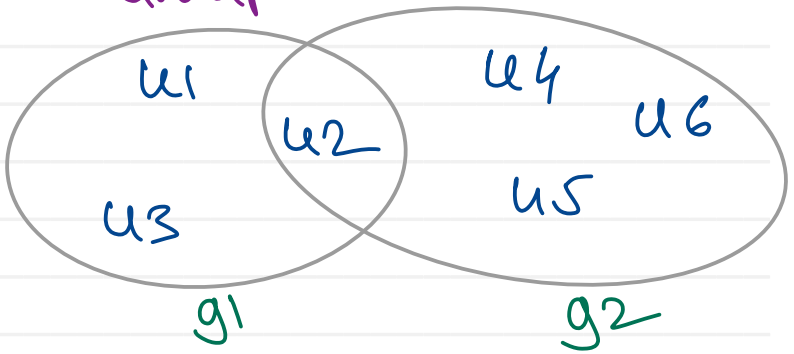
permissions | owner/user | group | timestamp

Types of file:
1. Regular file (-)
2. Directory file (d)
3. Link file (l)
4. Pipe file (p)
5. Socket file (s)
6. Char special file (c)
7. Block special file (b)

Permissions:
r - read
w - write
x - execute

levels:
1. user/owner
2. group
3. others

Group:



file name : move.txt
owner : u1 (rw-)
group : g1 (u2,u3) (rw-)
others : u4, u5, u6 (r--)

r = read , w = write , x - execute

+ : add
- : remove

chmod ± r/w/x filename

e.g. chmod -r file.txt
chmod +x file.txt

u - user
g - group
o - others

chmod u/g/o ± r/w/x filename

e.g. chmod u+x file.txt
chmod g-rw file.txt

0 - 000
7 - 111

Human readable format : rwx --- r--
Octal format : 704

rwx	---	r--
111	000	100
7	0	4

Existing perms : rwx --- r--
New perms : rw- rw- r--

rwx	---	r--
111	000	100
110	110	100
6	6	4

$\Rightarrow (664)_8$

chmod $(mode)_8$ filename

e.g. chmod 664 file.txt

+ : add
- : remove

r - read
w - write
x - execute

u - user
g - group
o - others
a - all

human format : rwx r-- r--

111 100 100

7 4 4

Numeric format : 744

remove execute of user
add write to group

existing : 111 100 100

new : 110 110 100

6 6 4

(664)

File = data + metadata

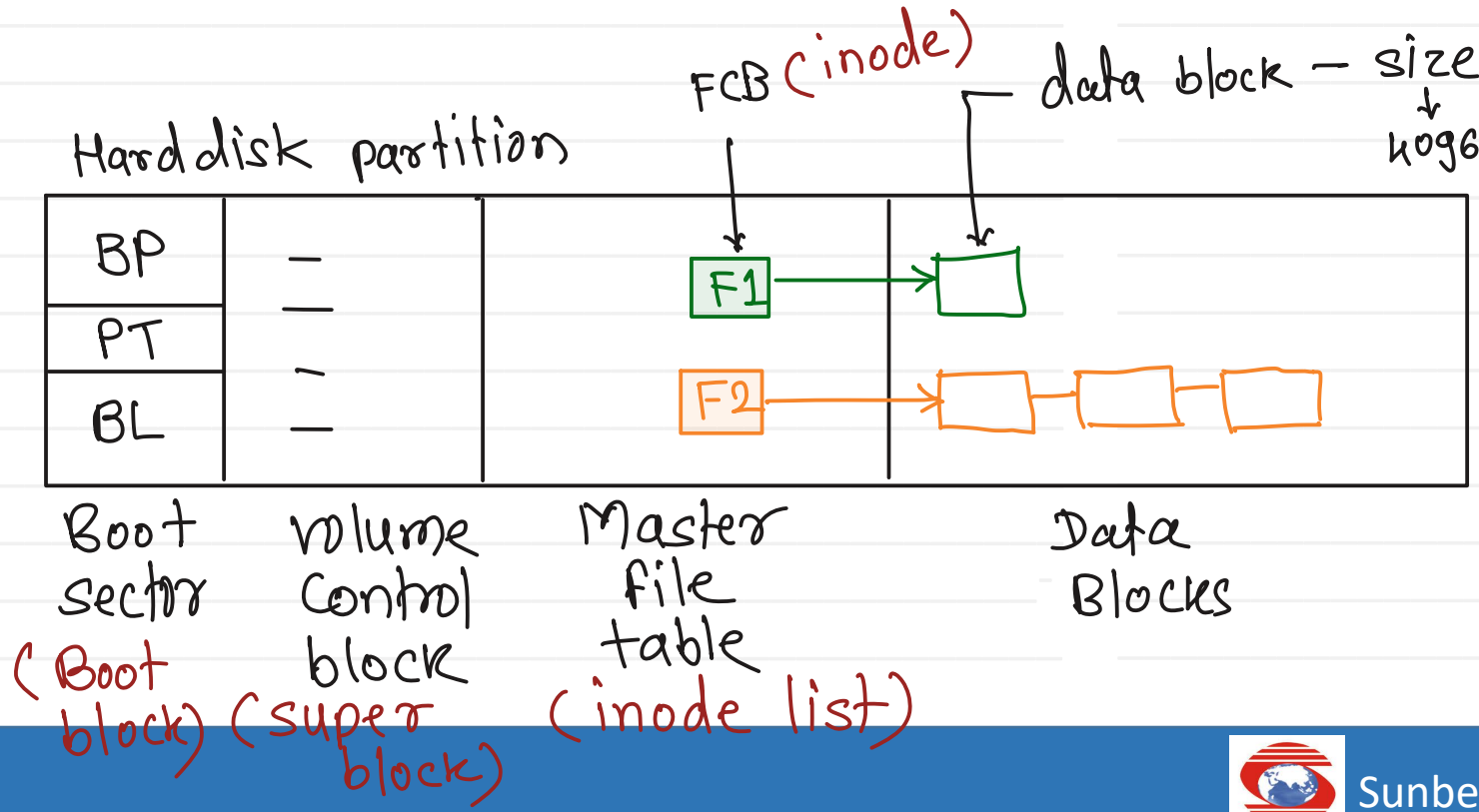
(Actual content of a file) → data block

(information about a file) → File Control Block (FCB)

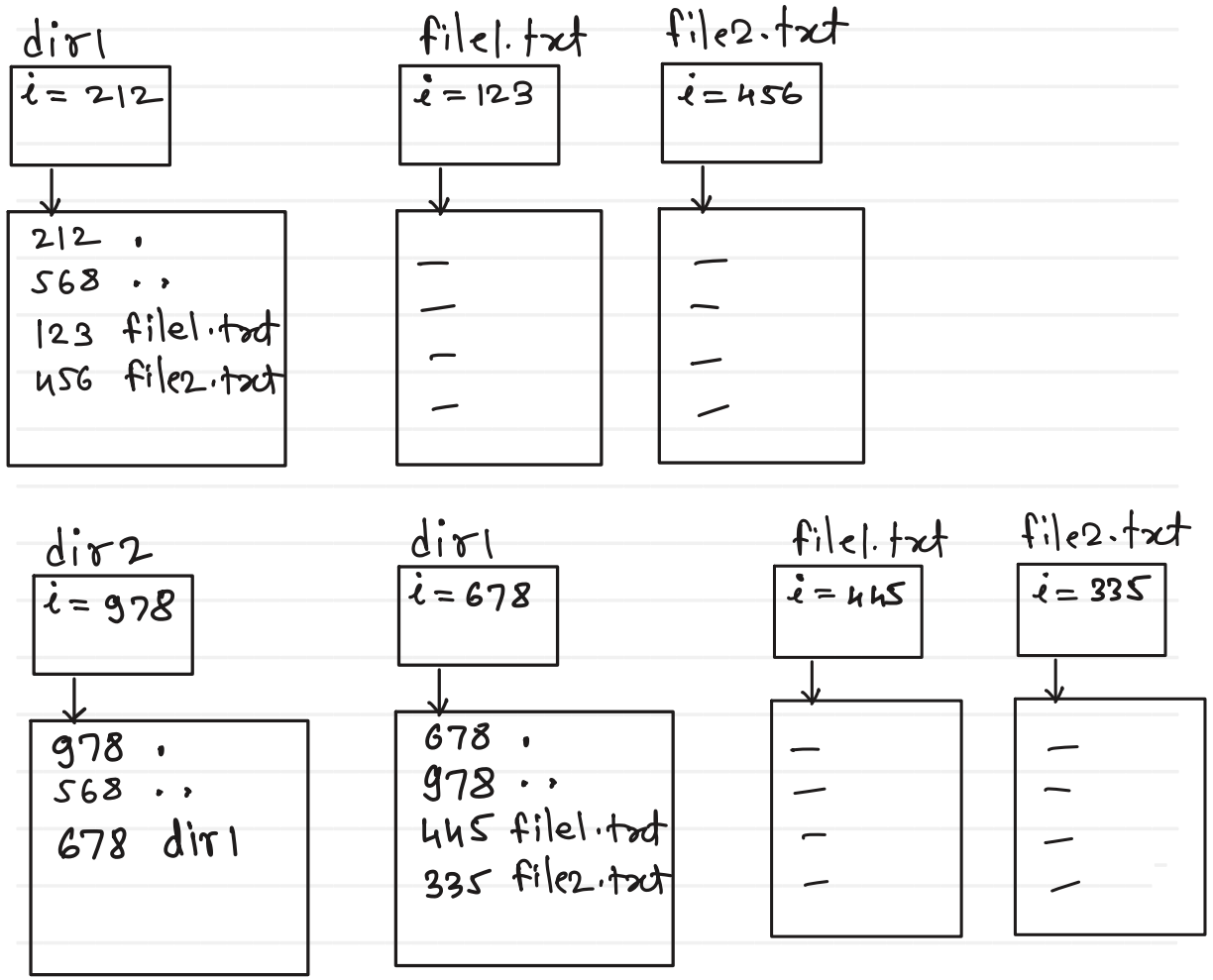
File system:

Organizing files on a partition

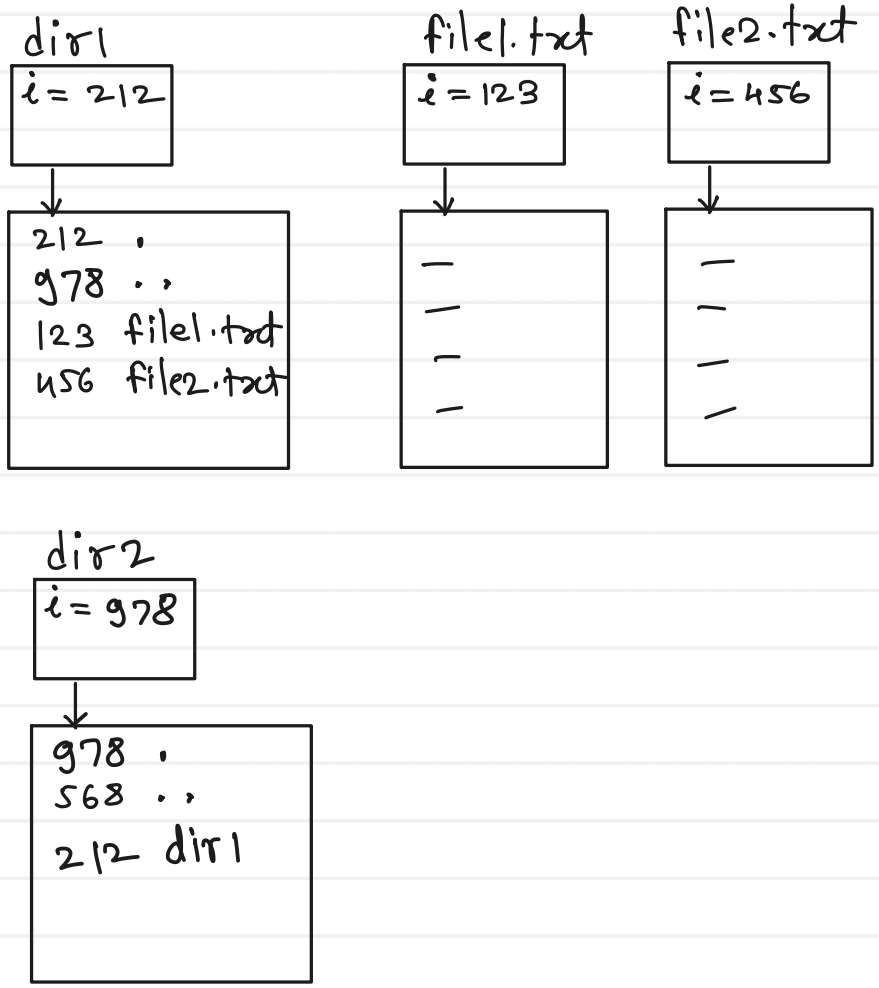
e.g. ext4, xfs, FAT32, NTFS, UFS, ZFS..



cp -r dir1 dir2

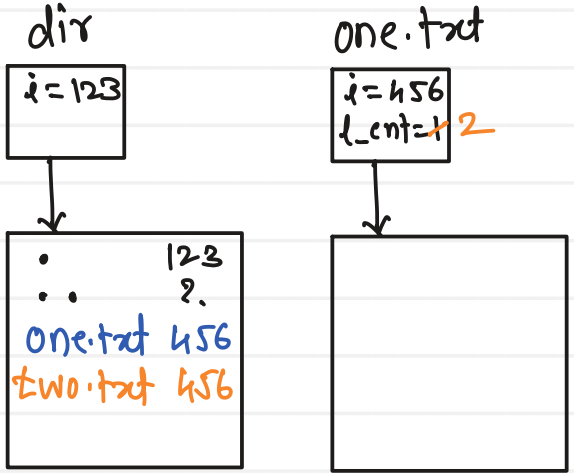


mv dir1 dir2



Hard link and Symbolic link

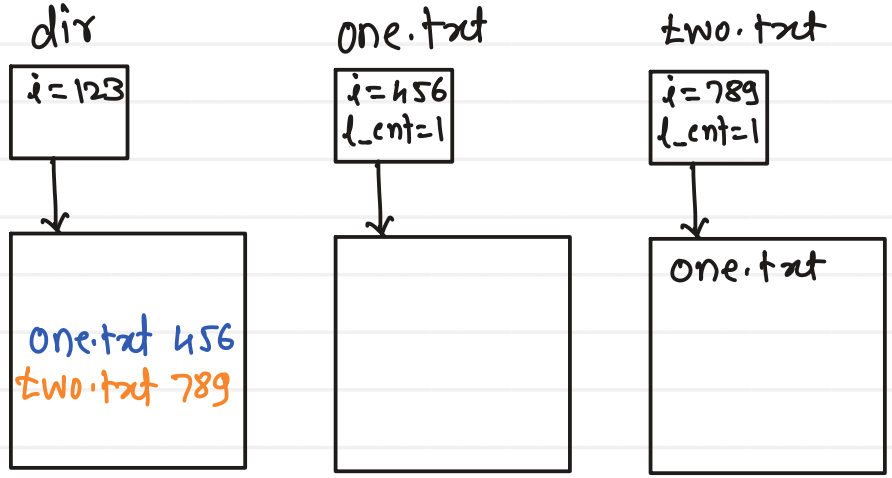
ln one.txt two.txt



- hard links are tightly coupled with target files
- user can not create hard link of directories because it may lead to infinite loop while accessing the directories
- hard links can not be created across file systems

Applications:
- to rename a file
- space efficient backups
- File level deduplication
- Robust file access

ln -s one.txt two.txt



- symbolic links are loosely coupled with target files
- symbolic links can be created for directories also.
- symbolic links can be created across files systems

Applications:
- to create shortcuts
- software versioning & control
- cross file system references
- Path simplification
- Directory linking



Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

ROM / Flash (UVROM / EEPROM)

→ firmware

- BIOS (Basic Input Output System) (MBR)
- UEFI (Unified Extensible Firmware Interface) (GPT)

Bootstrap
- POST / BIST :- checking all hardware components
- Bootstrap loader :- finding boottable device using device boot order
  - invoke bootloader
- minimal device drivers - (e.g. keyboad/mouse, monitor, HDD)

HDD / SSD - storage devices
- bootloader (GrUB)
- kernel image (vmlinuz)
- initial RAM file system (initrd / initramfs)
- program for first process (init / systemd)

Power ON

↓

BIOS / UEFI - CPU is hard wired to load firmware inside RAM

↓

POST - will check hardware components

↓

Bootstrap loader - will find bootable device & invoke bootloader

↓

Bootloader - will show menu to the user & load bootstrap program of user selected option

↓

Bootstrap program - loads kernel image & initramfs into RAM

↓

Kernel & initramfs - self extracted, creates first user space process (PID=1)

↓

systemd / init - created further processes & starting all required services

Linux Booting

- The processor checks for the BIOS/UEFI firmware and executes it
- BIOS/UEFI checks for bootable media from internal storage devices or peripherals like USB flash drives and DVD-ROMs. It locates a valid device to boot the system
- BIOS/UEFI loads the primary boot loader (probably GRUB2) from the MBR/GPT partition into memory. It also loads the partition table along with it.
- GRUB2 prompts the user to select an operating system to boot. If the user does not respond, then the default operating system is booted.
- The boot loader determines the kernel and locates the corresponding kernel binary
- It then uploads the respective initrd image into memory and transfers control of the boot process to the kernel
- The kernel configures the available hardware drivers, including processors, I/O subsystems, and storage devices. It decompresses the initrd image and mounts it to load the necessary drivers. If the system implemented any virtual devices, such as LVM or software RAID, then they are initialized
- The kernel mounts the main root partition and releases unused memory
- The systemd program runs to set up the user environment. It becomes process ID 1

Linux Booting

- The systemd program searches for the default.target file, which contains details about the services to start
- It mounts the file system based on the /etc/fstab file or .mount files and begins the process of starting services
- On most systems, the selection is either multi-user.target or graphical.target
- If graphical mode is selected, then a display manager starts and the login window is displayed on the screen
- The user enters a user name and password to log in to the system
- The system authenticates the user. If the user is valid, then various profile files are executed
- The shell starts, and the system is ready for the user

System and User Profiles

- As a sysadmin, you can use a few different files to set the system up the way your institution prefers
- Use /etc/profile to set system-wide environment variables and startup programs for new user shells
- Use /etc/bashrc to establish system-wide functions and aliases for new user shells
- The ~/.bash_profile sets user-specific environment variables for new Bash shells, and ~/.bashrc runs when noninteractive shells are launched
- The tilde character (~) represents the current user's home directory
- The system-wide files process first, and then the user-specific files are executed
- The user-specific configuration files take precedence over system files, allowing users to customize their environments to suit their needs



Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Storage

- **Storage** is used to **store data permanently or temporarily** in a computer system.

- Main purpose of **Storage** is to
  - Store OS (Linux, Windows)
  - Store user data (files, videos, code)
  - Store applications
  - Store logs and system data

Feature	Primary	Secondary
Speed	Very fast	Slower than RAM
Volatility	Volatile	Non Volatile
Size	Limited	Large
Cost	Expensive	Cheaper

- **Types of Storage**
  - **Primary storage (Temporary storage)**
    - RAM (DRAM)
    - Cache (L1, L2, L3)
    - ROM (Flash)
  - **Secondary storage (Permanent storage)**
    - HDD (Hard Disk Drive)
      - Spinning magnetic disks, usually inexpensive, large, and relatively slow
    - SSD (Solid State Drive)
      - Flash memory storage, usually expensive, fast, small, and shorter -lived
    - USB drives
      - USB - Connection protocol for various external devices
      - Removable storage media, connected via USB
      - Removable storage disk, connected via USB

Types of Secondary Storage

- Based on **technology used**, secondary storage is classified into:
  - **Magnetic Storage Devices**
    - Stores data using **magnetic fields** on a rotating surface
    - Disk surface is coated with **magnetic material**
    - Eg. HDD (Hard Disk Drive), Floppy, ZipDisk
  - **Electronic (Solid-State) Storage Devices**
    - Stores data using **electrical charges in semiconductor memory**
    - No moving part, Uses **flash memory (NAND)**
    - Eg. SSD (SATA SSD), NVMe SSD, USB Flash Drive
  - **Optical (Polycarbonate) Storage Devices**
    - Uses **laser light** to read/write data on a **polycarbonate disk**
    - Polycarbonate plastic Coated with reflective layer
    - Laser beam scans disk surface
    - Eg. CD, DVD, Blu-Ray

Comparison of Secondary Storage

Feature	Magnetic	Electronic	Optical
Technology	Magnetic field	Electric charge	Laser
Speed	Slow	Fast	Very slow
Moving parts	Yes	No	No
Cost	Cheap	Expensive	Very cheap
Capacity	Very high	High	Low
Reliability	Medium	High	Low

Types of Storage Interfaces

- **PATA (IDE)**- Parallel ATA / IDE
  - Old interface used for HDDs and CD drives
  - Parallel, Slow (~133 MB/s), Wide ribbon cable, Max 2 devices per cable
  - Naming Conventions - /dev/hda, /dev/hdb

- **SATA** – Serial ATA
  - Modern replacement for PATA, used for HDD, SATA SSD
  - Serial, speed Up to ~600 MB/s, Thin and flexible Cable, One device per port
  - Naming convention - /dev/sda, /dev/sdb

- **SCSI -** Small Computer System Interface
  - Used in **servers and enterprise systems**
  - High speed, more reliable, Multiple device
  - Naming convention - /dev/sdX

- **SAS -** Serial Attached SCSI
  - Advanced version of SCSI, Dual port, Better Performance than SATA
  - High speed, very high reliability, used in enterprise servers, data centers, high end storage devies

Types of Storage Interfaces

- **USB -** Universal Serial Bus
  - External storage interface, highly portable, moderate speed, plug and play
  - Versions - USB 2.0(~480 Mbps), USB 3.0(~5 Gbps), USB 3.1/3.2(faster)

- **PCIe (NVMe) -** NVMe Interface
  - High-speed interface for modern SSDs
  - Very high speed (GB/s), Very Low Latency, Direct communication with CPU
  - Naming convention - /dev/nvme0n1

- **Linux naming depends on driver, not on interface**
- In Linux many devices appears **/dev/sdX**, Because Linux uses **SCSI layer abstraction**
  - **/dev/sda -** A hard disk that uses the SCSI driver.
  - **/dev/nvme0n1** - The first hard disk on an NVM Express (NVMe) interface.
  - **/dev/hda** - The (legacy) IDE disk device type.
  - **/dev/vda** - A disk in a KVM virtual machine that uses the virtio disk driver
  - **/dev/xvda** - A disk in a Xen virtual machine that uses the Xen virtual disk driver

Partitioning Schemes

- Storage drives are divided into partitions for more efficient and organized use

- **MBR**
  - The master boot record (MBR) is the first physical sector on a storage drive and a type of partition structure
  - The MBR boot sector contains the boot loader that loads the operating system into memory
  - It also contains the partition table of the storage drive
  - MBR determines what sectors are available to each partition, as well as which partitions are bootable and which are not
  - It has three major disadvantages:
    - The maximum storage space of an MBR-partitioned drive is two terabytes
    - MBR-partitioned drives can have a maximum of four primary partitions
    - The boot data is stored all in one sector, which increases the risk of corruption

Partitioning Schemes

- **GPT**
  - The GUID Partition Table (GPT) is a successor to MBR that makes up for the latter's shortcomings
  - Like MBR, it is a partition structure, but it employs a more modern design and is part of the UEFI standard
  - Every partition on a drive is assigned a globally unique identifier—a GUID—to distinguish it from every other partition on (theoretically) every drive
  - The storage space and partition number maximums are so large that they are not currently achievable, and any limitations are going to be imposed by the file system type or operating system kernel, rather than GPT itself.
  - GPT also has the advantage of storing its boot data in multiple locations on a drive to enhance redundancy
  - If the primary location is corrupted, GPT can leverage one of the other copies to restore the boot data

MBR Vs GPT

MBR = Master Boot Record

- Old partitioning scheme
- Introduced in early systems (BIOS-based)

- Structure of MBR
  - [Bootloader | Partition Table | Signature]
  - Stored in first sector (512 bytes) of disk
  - Contains: Bootloader, Partition table

- Limitations
  - Max disk size: 2 TB
  - Max partitions: 4 primary partitions
  - To create more: Use extended + logical partitions

- BIOS boot
- Bootloader in MBR

GPT = GUID Partition Table

- Modern partitioning scheme
- Part of UEFI system

- Structure of GPT
  - [Protective MBR | GPT Header | Partition Entries | Backup Header]

- Supports large disks (>2 TB) 9.2 ZB
- Stores multiple copies of partition table (backup)
- Up to 128 partitions (or more)
- Better reliability (backup + CRC checks)

- UEFI boot
- Bootloader in EFI partition

MBR Vs GPT

Feature	MBR	GPT
Full Form	Master Boot Record	GUID Partition Table
Age	Old	Modern
Max Disk Size	2 TB	> 2 TB
Partitions	4 primary	128+
Boot Mode	BIOS	UEFI
Backup	No	Yes
Reliability	Low	High
Partition Info	Single location	Multiple copies

Partitioning command

Command	Use
`fdisk`	Basic partitioning (MBR)
`parted`	Advanced (GPT, large disks)
`lsblk`	View disks
`blkid`	FS info
`partprobe`	Reload changes
`cfdisk`	UI-based
`sfdisk`	Scripting

Partitioning command

- **fdisk (Most Common - MBR)**
  - **Command > fdisk /dev/sda**
  - **Options**
    - m - Help
    - p - Print partition table
    - n - New partition
    - d - Delete partition
    - t - Change partition type
    - w - Write changes (save)
    - q - Quit without saving
    - l - List partition types

- **parted (Advanced - GPT Support)**
  - **Command > parted /dev/sda**
  - **Commands**
    - print - Show partitions
    - mklabel gpt - Create GPT table
    - mkpart - Create partition
    - rm - Delete partition
    - quit - Exit

Creating File System

- After creating the partitions on a new drive, you must add a filesystem to organize the storage of the actual data
- Different filesystems have different benefits, and there are many of them
- In general, most Linux distributions use either the ext4 or XFS filesystems.
- **ext4 Filesystem**
  - The ext4 filesystem is the one of the oldest file systems
  - It provides journaling and is recognized by most distributions.
  - To place the ext4 filesystem on the second partition on the new disk, type:
    - **mkfs.ext4 <partition>**
  - The mkfs also recognizes a slightly different syntax.
    - **mkfs -t ext4 <partition>**

- **XFS Filesystem**
  - While the ext4 filesystem has been the default for many Linux distributions, it is very common to find newer systems relying on XFS
  - XFS has many modern advantages over ext4 and relatively few disadvantages
  - XFS recognizes a larger partition size
  - XFS recognizes a larger file size
  - XFS avoids inode exhaustion

File System Mounting - Manual

- Now that you have installed the drive, created the partitions, and placed the filesystem, the storage capacity is ready for use
- **mount**
  - The term for attaching storage to the FHS is referred to as "mounting," and the command is mount
  - To mount the partition, first create a directory
  - The directory is referred to as a mount point
  - Use the mount command to mount a fs to a directory
    - > **sudo mount <partition> <mount point>**

- **findmnt**
  - Find all the mounted file system

- **umount**
  - The logical reverse of this is to detach or unmount the storage area, and the related command is umount
  - Use umount command to unmount the mounted partition
    - > **umount <partition>**

Persistent Mounting

- /etc/fstab is the main configuration file to persistently mount partitions
- /etc/fstab content is used to generate system mounts by the system-fstab-generator utility

```
/dev/disk/by-uuid/25fd6706-979a-4e16-b52d-8af2a2331d5f /home ext4 defaults 0 0
```

- Every entry contains:
  - Column 1: device to be mounted
  - Column 2: mount point
  - Column 3: file system
  - Column 4: extra mount options
  - Column 5 and 6: always 0

- To update systemd, make sure to use systemctl daemon-reload after editing /etc/fstab
- Use mount –a to check the entries are working from fstab file
- Note: Do not persist mount on /mnt as it is used to mount a FS temporarily

LVM

- **Logical Volume Manager**

- It is a **storage management layer** in Linux that sits between Physical Disks and Filesystem
- Instead of using disks/partitions directly, LVM creates a **virtual layer**:
  - Disk → LVM → Logical Volume → Filesystem
- Traditional partitioning:
  - Disk → Partition → Filesystem
  - Limitations:
    - Fixed size partitions
    - Difficult to resize
    - Cannot combine multiple disks easily
- Solution → LVM
  - Resize partitions dynamically
  - Combine multiple disks
  - Better storage utilization

LVM Provides

- **Flexibility**
  - Increase size anytime
  - Reduce size (carefully)
- **Scalability**
  - Add new disk → extend storage
- **Abstraction**
  - User does NOT deal with raw disks directly

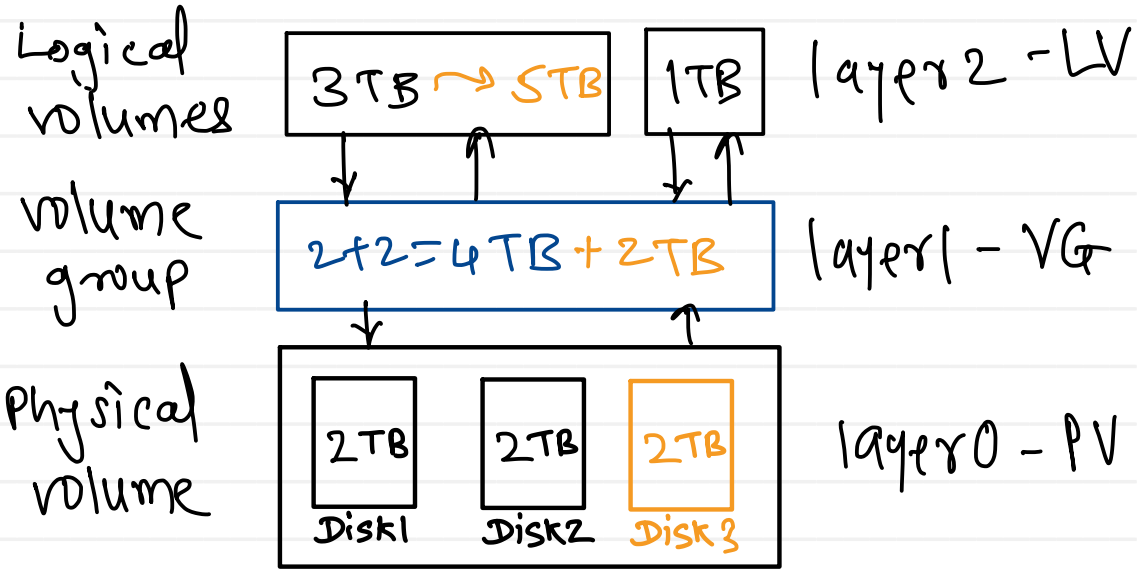
LVM - Layers

- Logical Volume Manager functions based on three configuration layers for storage

- The first layer is one or more **Physical Volumes (PV)**
  - PVs are drives or partitions allocated to LVM
- The next layer represents the aggregated storage capacity of any designated PVs that are added to the **Volume Group (VG)**
- The top layer consists of one or more **Logical Volumes (LV)**, which are treated by the system as if they were standard partitions
  - In reality, they are carved space from VGs that may store data on multiple PVs

- The /dev/mapper/ directory contains all of the logical volumes on the system that are managed by LVM
  - /dev/mapper/<volume group name>-<logical volume name>





LVM Advantages

- Dynamically create, delete, and resize volumes without having to reboot the system
- Day-to-day management of volumes is easier once everything is set up
- Map multiple logical volumes across multiple physical devices
- A logical volume can exceed the size of any one physical device (as long as it doesn't exceed the total size of devices in the volume group)
- Create virtual snapshots of each logical volume so you can quickly and easily revert a volume to a specific state

Commands for configuring LVM

- **Physical Volume**
  - pvcreate: creates new physical volume
  - pvscan: scans existing physical volumes
  - pvdisplay: displays attributes of selected physical volume
  - pvs: shows existing physical volumes

- **Volume Group**
  - vgcreate: creates mew volume group
  - vgscan: scans existing volume groups
  - vgdisplay: displays attributes of selected volume group
  - vgs: shows existing volume groups

- **Logical Volume**
  - lvcreate: created new logical volumes
  - lvscan: scans existing logical volumes
  - lvdisplay: displays attributes existing logical volumes
  - lvvs: shows existing logical volumes

Abbrevations

- HDD - Hard Disk Drive
- SSD - Solid State Drive
- USB - Universal Serial Bus
- CD - Compact Disk
- DVD - Digital Versatile Disc
- ATA - AT(Advanced Technology) Attachmnet
- PATA / IDE - Parallel AT Attachment / Integrated Device Electronics
- SATA - Serial AT Attachment
- SCSI - Small Computer System Interface
- SAS - Serial Attached SCSI
- PCIe - Peripheral Component Interface - Express
- NVMe - Non-Volatile Memory Express
- KVM - Kernel-based Virtual
- MBR - Master Boot Record
- GPT - GUID Partition Table
- GUID - Globally Unique Identifier
- BIOS - Basic Input Output System
- UEFI - Unified Extensible Firmware Interface
- GrUB - Grand Unified Bootloader
- XFS - eXtended File System
- FHS - Filesystem Hierarchy Standard
- LVM - Logical Volume Manager
- PV - Physical Volume
- VG - Volume Group
- LV - Logical Volume

Booting

- Process from computer power on to OS startup.

Bootable device

- Stoarage device (disk, pen drive, CD/DVD, etc.) whose boot block contains bootstrap program, is said to be Bootable device.

Bootstrap program

- Bootstrap program can load OS kernel into RAM and start its execution.
- Bootstrap program is different for each OS each version.
- Bootstrap is located in first sector (512 bytes) of a disk/partition.

Bootloader program

- Bootloader program can be configured to show multiple boot options to end user.
- Depending on user selection, it runs Bootstrap program of corresponding OS.
- There are many important bootloader programs.

SUNBEAM

    - ntldr (Before Windows Vista) --> boot.ini
    - "bootmgr" (Windows Vista Onwards) --> bcd (boot config data)
        - (admin command prompt) cmd> bcdedit
    - LiLo --> Linux Loader
    - "GrUB" (Grand Unified Boot Loader) --> menu.lst or grub.cfg
    - BTX (BooT eXtended) --> BSD Unix
    - SILo (Sparc Interactive Loader) --> Solaris
    - Bootcamp --> Mac OS X
    - "uBoot" --> Linux bootloader for Embedded
- The Bootloader has some config file associated with it.

Bootstrap loader

- The set of programs fixed in Base ROM (Motherboard) is called as "Firmware".
    - BIOS (Basic Input Output System) -- Firmware developed for PC by IBM + MS.
    - EFI (Extensible Firmware Interface) -- Firmware developed for PC by Intel + HP.
- Bootstrap loader is a program from Base ROM.
- It find the bootable device as per "boot device priority" in the BIOS setup.
- Once bootable device is found, it starts its bootloader.

Booting steps

- Computer power on.
- Load firmware (BIOS or EFI) programs from base ROM into RAM.
- Run POST/BIST (from firmware).
- Run Bootstrap loader (from firmware) to find bootable device.
- Bootstrap loader loads bootloader program from bootable device.
- Bootloader program shows multiple options to end user and user select one of them.
- Bootloader program run corresponding bootstrap program.
- Bootstrap program load OS kernel into main memory and OS boot
- Startup Process
    - Linux kernel access Root file system.
    - systemd/init (sbin directory) process is executed (pid=1).
    - systemd is designed to start (independent) services parallely. In init services were started sequentially.
    - It starts multiple child processes as per configred run-level.
    - Runlevels are like "checkpoints"/"states".
- Runlevels
    - 1 - Single user mode (Used for failsafe/rescue)
    - 2 - Multi user mode (User login enabled, User files accessible)

SUNBEAM

    - 3 - Networking (Multiuser with Networking on CLI) -- multi-user.target
    - 4 - Reserved (unused)
    - 5 - Graphical user interface -- graphical.target
    - https://likegeeks.com/linux-runlevels/

- Linux commands

    - terminal> runlevel
    - terminal> init 0 --> shutdown
    - terminal> init 6 --> reboot
    - terminal> init 3 --> go to runlevel 3 (GUI will be stopped if already started)
    - terminal> init 5 --> go to runlevel 5 (GUI will be started)
    - terminal> startx --> start GUI (give this command from runlevel 3)

BIOS = Basic Input Output System

- It is a standard for firmware interfaces stored on a computer motherboard's ROM chip
- The BIOS firmware runs when the computer powers on, enabling it to test the various hardware components in a computer and run the boot loader to start the operating system
- The BIOS has access to the ports used by basic hardware input devices like a mouse and keyboard. Users can also load up a BIOS interface instead of an operating system to make various hardware-level changes
- BIOS was the dominant standard for home and enterprise computers for several decades
- Functions of BIOS
    - Initialize hardware (CPU, RAM, disk)
    - Perform POST (Power-On Self Test)
    - Load bootloader from disk

UEFI = Unified Extensible Firmware Interface

- Modern replacement for BIOS
- It is newer firmware technology that has largely replaced BIOS by bringing with it several key advantages
- UEFI runs faster than BIOS, can operate within a greater amount of memory, can access storage drives of currently unattainable sizes, can access more hardware types, and has improved security protections
- Most modern motherboards, as well as the pre-assembled PCs that use them, ship with UEFI. The secure boot features of UEFI are critical
- Like BIOS, UEFI provides an environment to execute a boot loader and ultimately start up the operating system
- Functions of UEFI
    - Initialize hardware
    - Load OS
    - Provide advanced features
- EFI Partition
    - Special partition:/boot/efi
    - Stores bootloader files
    - Used by UEFI
- Features

SUNBEAM

- Secure Boot
- Faster startup
- Better hardware support
- Network boot capability

GrUB

- The boot loader uses three main components that work together to load the operating system in stages:
  - The boot sector program is loaded by a boot environment on startup and has a fixed size of 512 bytes. Its main function is to load the second-stage boot loader; however, it can also load another sector or a kernel
  - The second-stage boot loader loads the operating system and contains a kernel loader
  - Finally, the boot loader installer controls the installation of drive sectors and can only be run when booting from a drive. It coordinates the activities of the boot sector and the boot loader

SysV init – Boot Process (Step-by-Step)

- Step 1: System Power ON
  - BIOS/UEFI runs
  - Hardware is initialized
- Step 2: Bootloader Executes
  - Bootloader (like GRUB) loads Linux kernel into memory
- Step 3: Kernel Initialization
  - Kernel initializes:
    - CPU
    - Memory
    - Devices
  - After initialization:
    - Kernel starts init process
- Step 4: Init Process Starts (PID = 1)
  - Kernel executes:
    - /sbin/init
- Step 5: Read Configuration File
  - /etc/inittab
  - Init reads:
    - Default runlevel
    - What to execute
- Step 6: Determine Default Runlevel
  - Example:
    - id:3:initdefault:
  - System enters:
    - Runlevel 3 (multi-user CLI)
- Step 7: Execute Startup Scripts
  - Based on runlevel, init executes scripts from:
    - /etc/rc.d/rcX.d/
  - (X = runlevel)

SUNBEAM

- Step 8: Script Execution Order
  - Scripts are executed sequentially
  - Naming:
    - S10network
    - S20ssh
    - K10httpd
- Step 9: Start Services
  - Examples:
    - Network
    - SSH
    - Database
  - One by one (no parallel execution)
- Step 10: System Ready
  - After all scripts executed:
  - Login prompt appears
  - System ready for user

systemd – Boot Process (Step-by-Step)

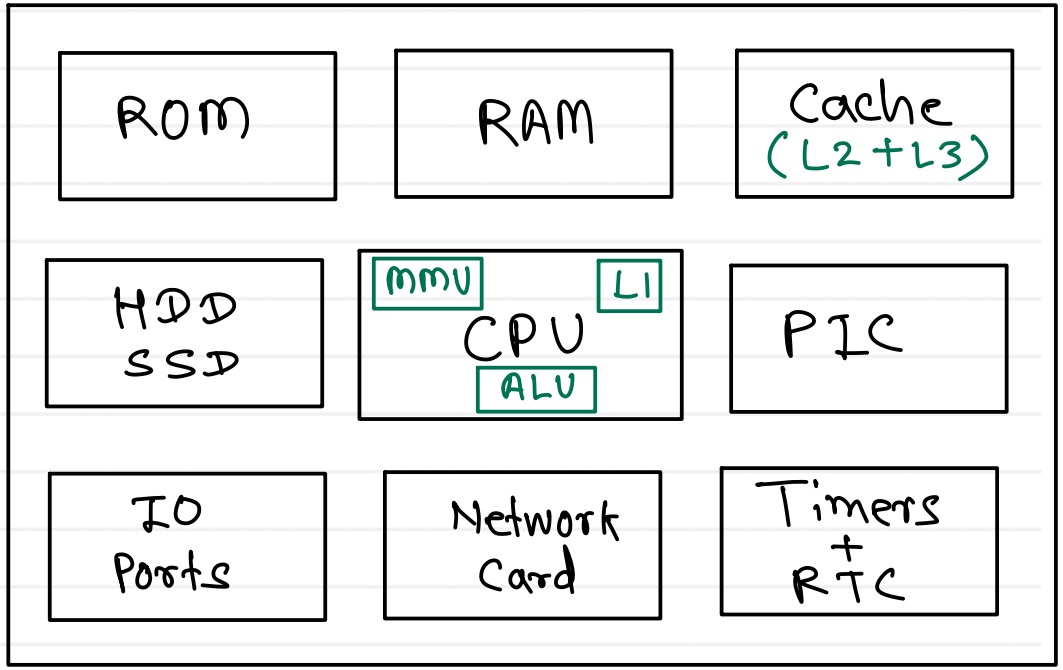
- Step 1: System Power ON
  - BIOS/UEFI starts
  - Hardware initialization
- Step 2: Bootloader Executes
  - Bootloader (GRUB) loads Linux kernel into memory
- Step 3: Kernel Initialization
  - Kernel initializes:
    - CPU
    - Memory
    - Devices
  - After initialization
    - Kernel starts systemd
- Step 4: systemd Starts (PID = 1)
  - Kernel executes:
    - /usr/lib/systemd/systemd
  - This is the first user-space process
- Step 5: Load Default Target
  - systemd reads default target from:
    - /etc/systemd/system/default.target
- Step 6: Identify Target
  - Example:
    - multi-user.target
  - Equivalent to:
    - Runlevel 3 (CLI mode)
- Step 7: Load Unit Files
  - systemd loads unit files from:
    - /etc/systemd/system/
    - /lib/systemd/system/

SUNBEAM

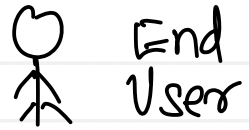
- Step 8: Resolve Dependencies
  - systemd checks:
    - Which services depend on others
    - Which can run in parallel
- Step 9: Parallel Service Startup
  - Unlike SysV:
    - systemd starts services in parallel
  - Faster boot
  - Efficient resource usage
- Step 10: Activate Required Units
  - Types:
    - .service → services
    - .mount → filesystems
    - .socket → communication
- Step 11: Mount Filesystems
  - systemd mounts required filesystems automatically
  - Uses configuration like:
    - /etc/fstab
- Step 12: Start System Services
  - Examples:
    - Network
    - SSH
    - Logging
- Step 13: System Ready
  - Final state:
    - Target reached
    - Login prompt / GUI appears

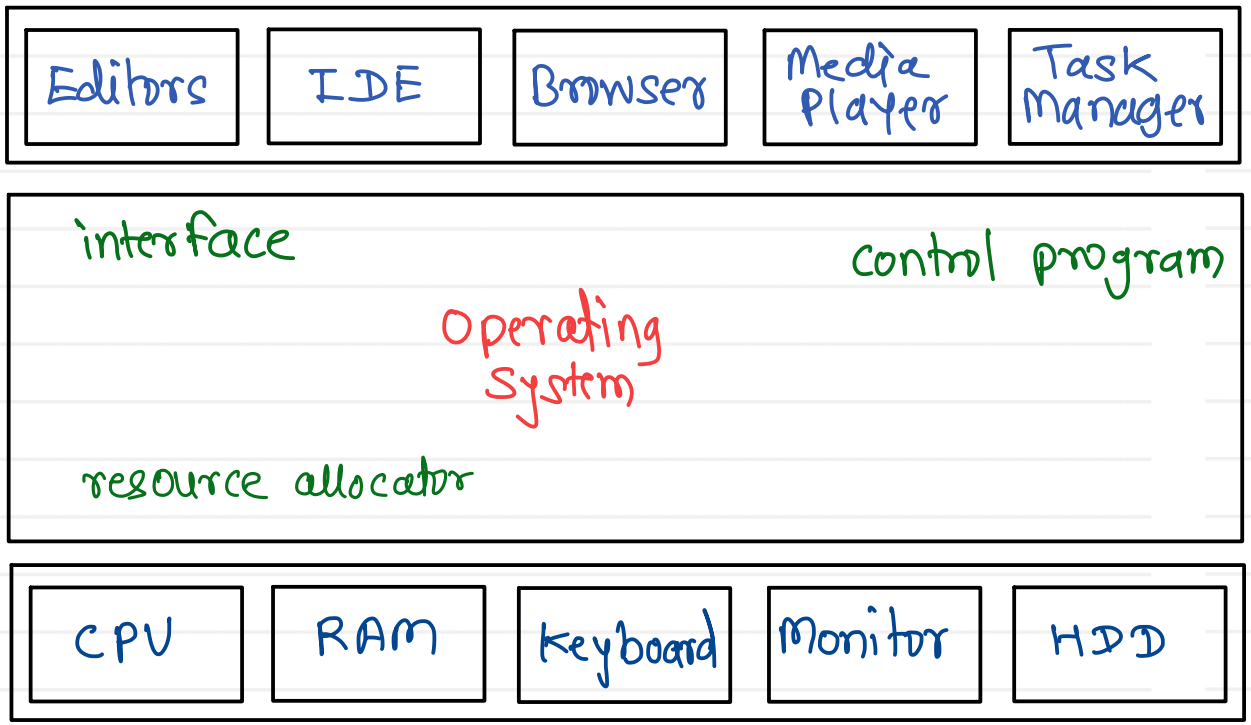
SUNBEAM

Computer Architecture



Operating system

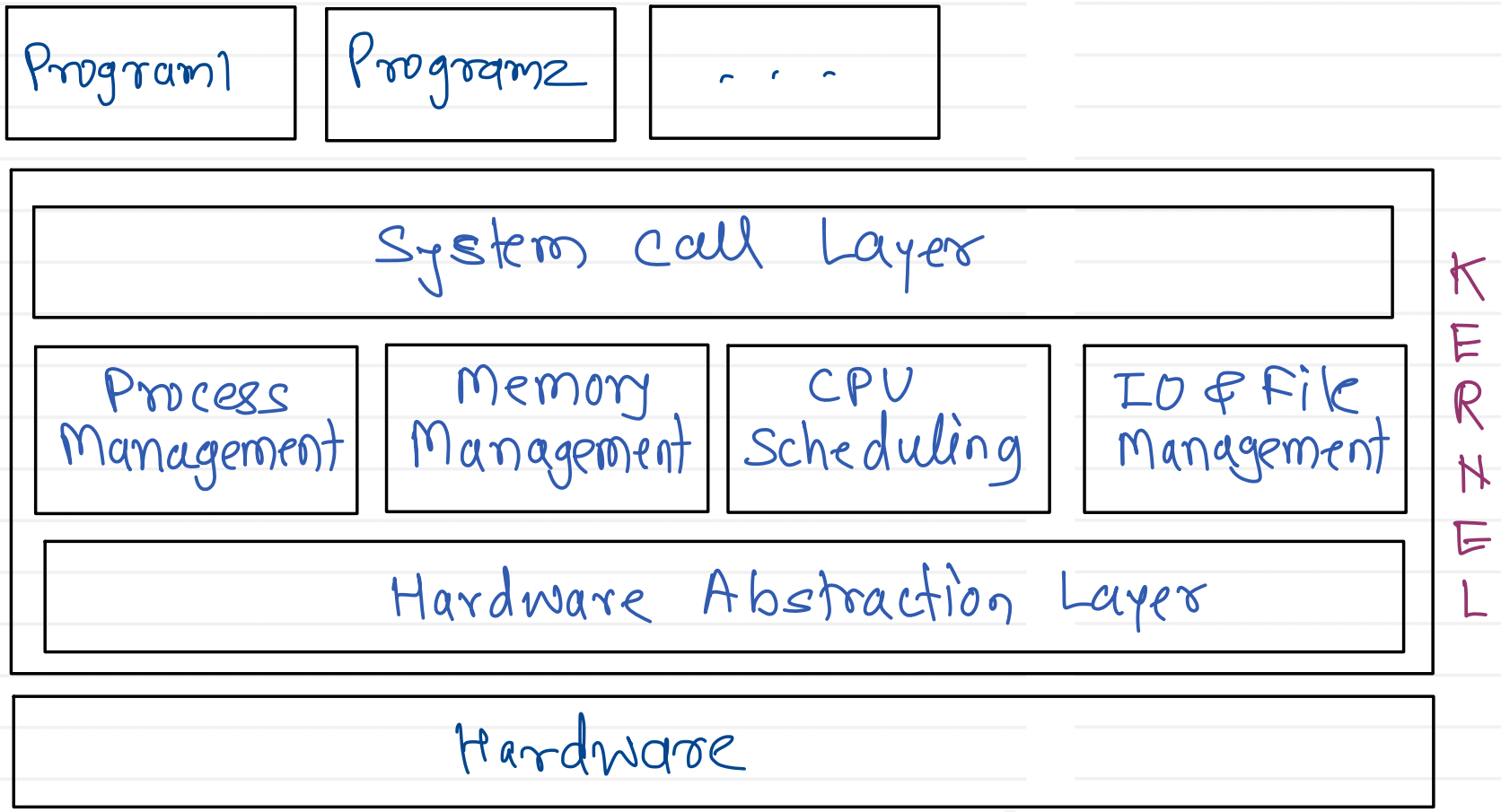




- interface betn end-user and hardware
- interface betn application softwares and hardware
- Control program which controls execution of programs running on the top of it.
- resource allocator/manager which manages limited hardware resources.

CD/DVD/ISO : Core OS (Kernel) + System Utilities + Appln S/W

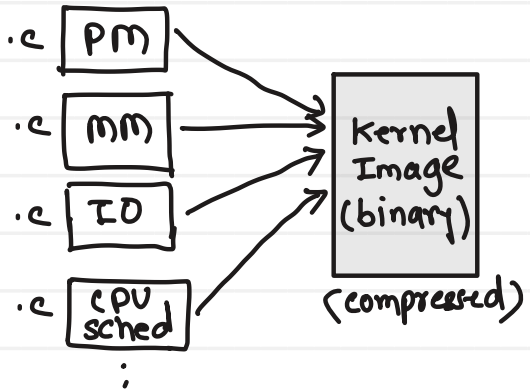
OS(Kernel) functions



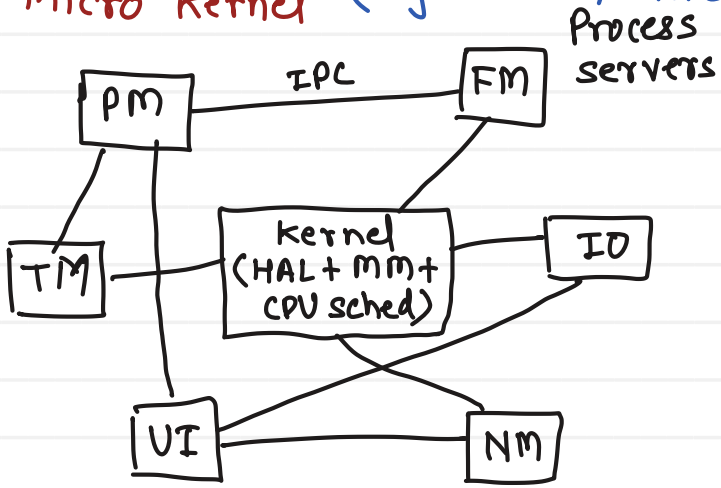
- Networking
- User Interfacing
- Security & protection

Types of kernel

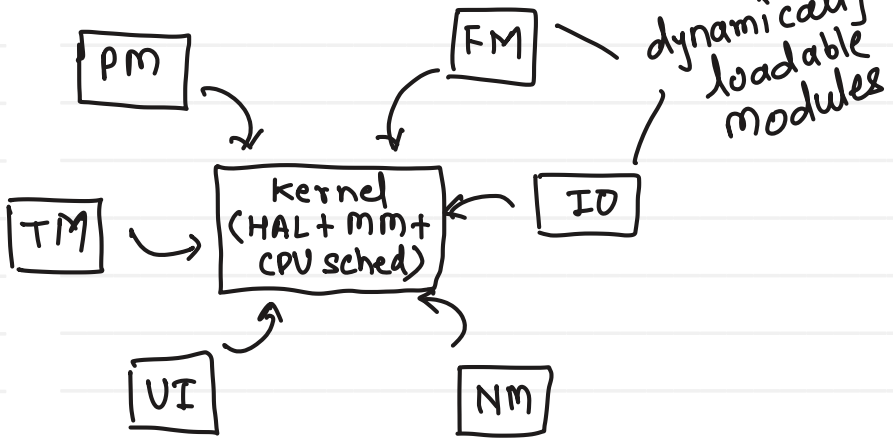
1. Monolithic kernel (BSD UNIX)



2. Micro kernel (Symbian, MACH)



3. Modular kernel (Windows - ntosldr.exe)



4. Nano kernel (FreeRTOS)

limited / required functionalities will be kept.

5. Hybrid kernel

- Combination of two or more kernels.

(MAC OS - Darwin (BSD UNIX + MACH)

Linux kernel

Linux Kernel = (Monolithic) Static Component + (Modular) dynamic component + (Micro) VI

Static Component:
1. Process Management
2. CPU scheduling
3. Memory Management
4. File & IO Management
5. Hardware Abstraction
6. System calls
⋮

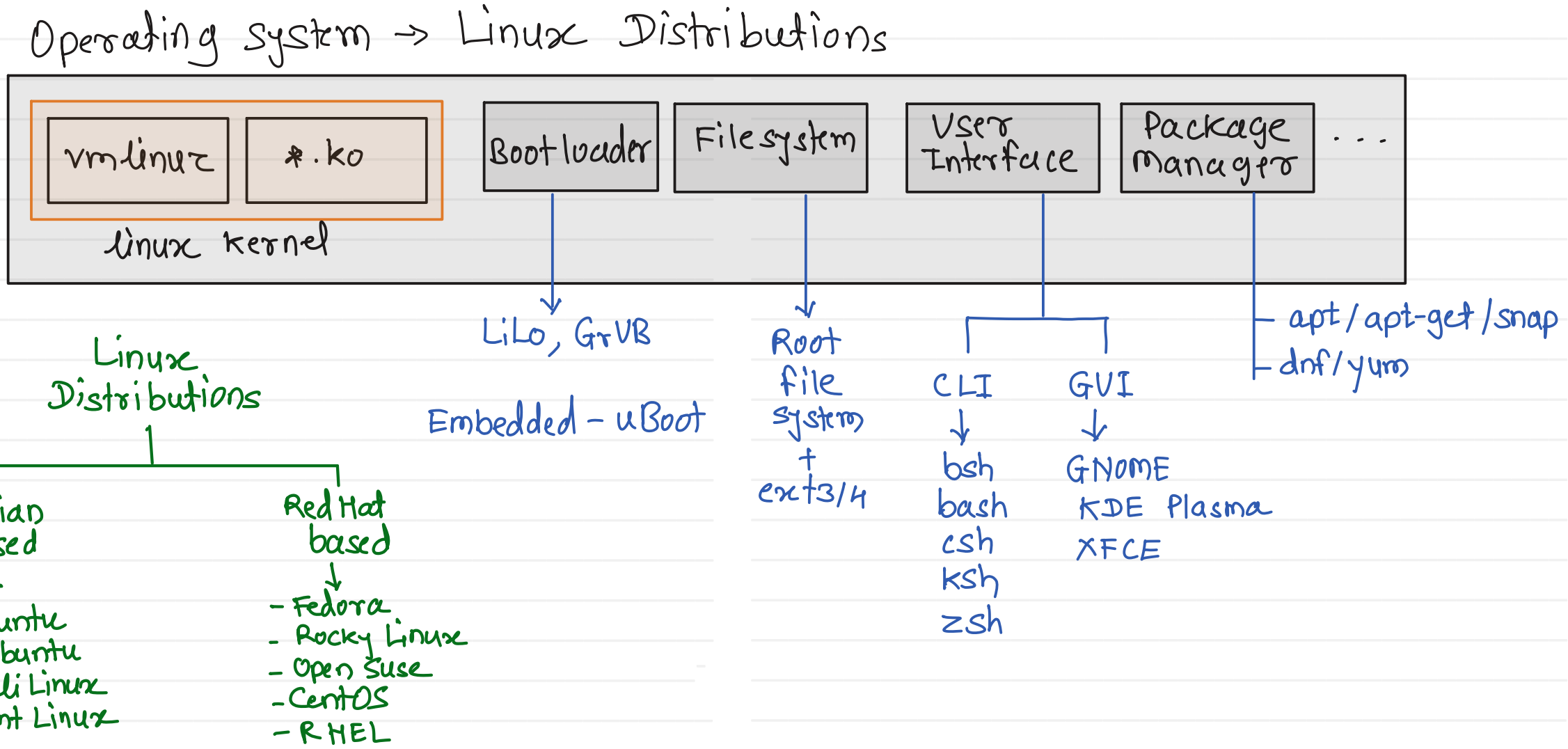
Kernel image (binary)
↓
vmlinuz

dynamic component:
1. File system managers
2. Device drivers
⋮

dynamically loadable modules
(Kernel Objects - .ko)

VI:
process servers
↓
xServer

Linux distributions



User Interfacing

CLI (Command Line Interface)

- is provided with the help of shell.
- shell is a program which acts as a intermediate between OS & user
- shell is also called as command interpretor.

Windows : cmd.exe , powershell.exe
Linux : bsh(sh), bash, csh, ksh, zsh

↓

Bourne Again Shell
(Default shell)

GUI (Graphical User Interface)

- provided as seperate process servers
- xserver

Windows : explorer.exe
Linux :

1. GNOME

(GTK) (GNU Network Object Model Enviroment)

2. KDE plasma

(Qt) (K Desktop Environment)

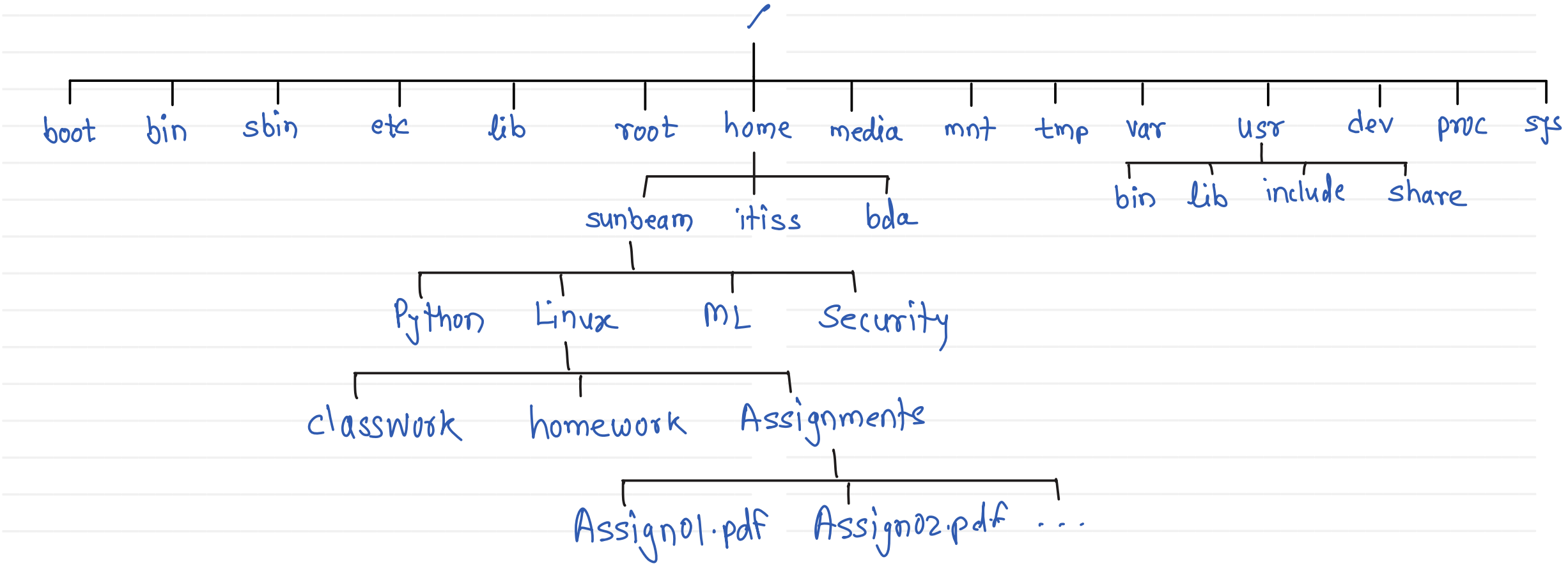
3. XFCE

(XForm) (XForms Common Environment)

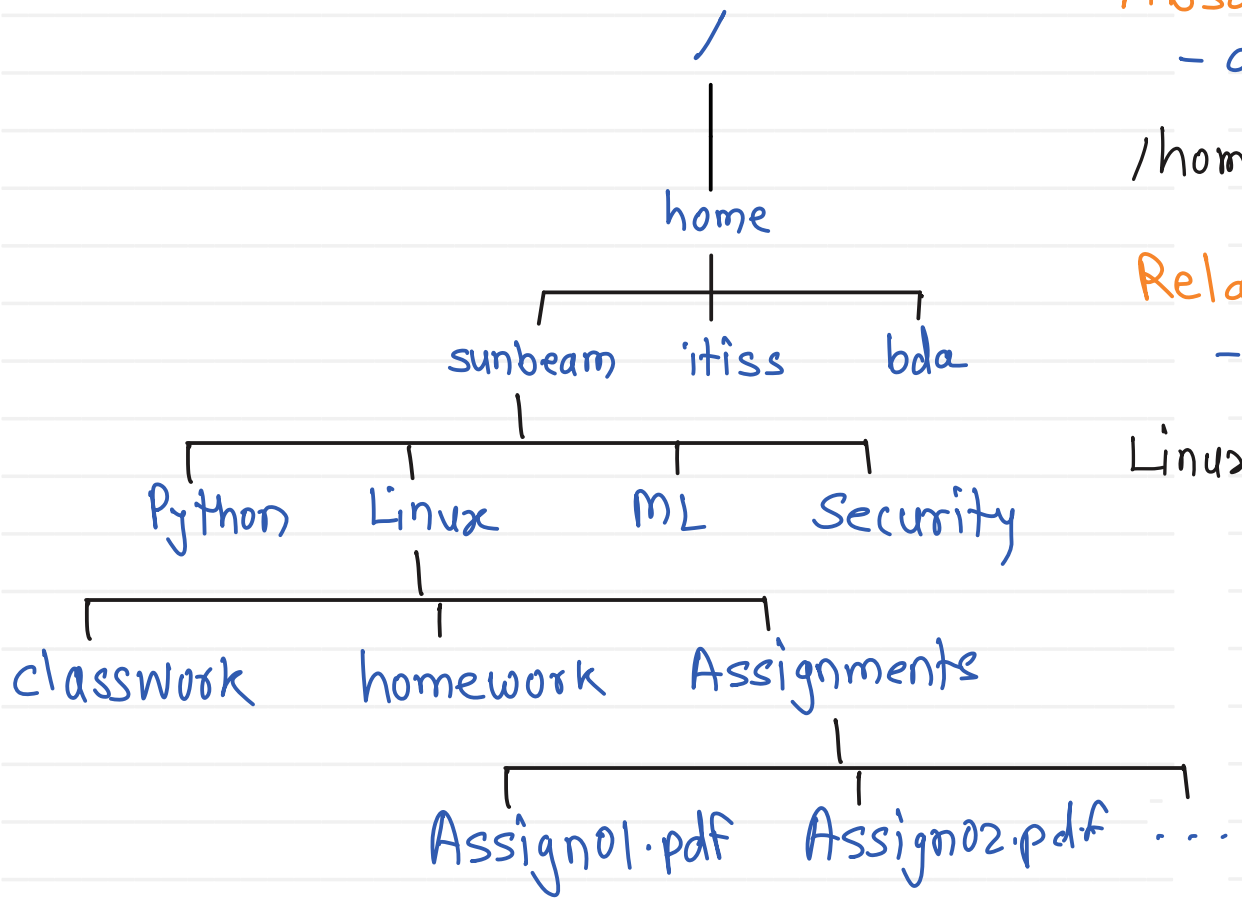
↑

tool kits (libraries) to create GUI

Linux file structure



Types of path



Absolute path:
- always starts with '/'

/home/sunbeam/Linux/Assignments/Assign01.pdf

Relative path:
- always starts with respect to working dir

Linux/Assignments/Assign01.pdf

Linux Network Related Commands

1. `ip`

Description

- `ip` is a modern Linux command used to inspect and configure network-related information.
- It can be used to manage:
  - Network interfaces
  - IP addresses
  - Routing tables
  - ARP/neighbor information
- It is part of the `iproute2` package.
- It is commonly used instead of older commands such as `ifconfig` and `route`.

Syntax

```
ip [OPTIONS] OBJECT COMMAND
```

Common form:

```
ip OBJECT COMMAND
```

SUNBEAM

`ip address`

Displays IP address information of network interfaces.

Syntax

```
ip address
ip addr
ip a
```

Example output:

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 192.168.1.10/24
    inet6 fe80::1234/64
```

Important Output

- `eth0`
  - Network interface name.
- `UP`
  - Interface is administratively enabled.
- `LOWER_UP`
  - Physical/link layer is active.
- `inet 192.168.1.10/24`
  - IPv4 address.
  - `/24` represents the network prefix length.
- `inet6 fe80::1234/64`

SUNBEAM

- IPv6 address.

---

ip link

Displays network interface information.

Syntax

```
ip link
ip l
```

Example output:

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
```

Important Information

- Interface name
- Interface state
- MAC address
- MTU
- Link status
- Interface flags

---

ip route

Displays the routing table.

Syntax

SUNBEAM

```
ip route
```

Example output:

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
```

Important Output

- `default`
  - Default route.
  - Used when no more specific route exists.
- `via 192.168.1.1`
  - Gateway used for the route.
- `dev eth0`
  - Network interface used for the route.
- `src 192.168.1.10`
  - Source IP address used for the route.

`ip neigh`

Displays the neighbor table.

For IPv4, this is commonly associated with ARP information.

SUNBEAM

Syntax

```
ip neigh
```

Example output:

```
192.168.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
```

Important Output

- `192.168.1.1`
  - IP address of the neighbor.
- `lladdr aa:bb:cc:dd:ee:ff`
  - MAC address of the neighbor.
- `REACHABLE`
  - Neighbor is considered reachable.

Other possible states:

- `STALE`
- `DELAY`
- `PROBE`
- `FAILED`

---

2. `ping`

SUNBEAM

Description

- `ping` is used to test network reachability between two hosts.
- It normally uses ICMP Echo Request and Echo Reply for IPv4.
- It can be used to check:
  - Host reachability
  - Round-trip time
  - Packet loss
- A successful `ping` does not guarantee that a particular service is available.
- For example, a system can respond to `ping` while SSH port `22` is closed.

Syntax

```
ping [OPTIONS] destination
```

Example

```
ping 192.168.1.10
```

or:

```
ping google.com
```

Important Options

`-c`

SUNBEAM

Specifies the number of packets.

```
ping -c 4 192.168.1.10
```

Sends 4 packets and then exits.

Without `-c`, Linux `ping` normally continues until stopped using:

```
Ctrl + C
```

`-i`

Specifies the interval between packets.

```
ping -i 2 192.168.1.10
```

Sends approximately one packet every 2 seconds.

`-W`

Specifies the timeout for a response.

```
ping -W 2 192.168.1.10
```

Waits approximately 2 seconds for a response.

SUNBEAM

`-s`

Specifies packet size.

```
ping -s 1000 192.168.1.10
```

Sends an ICMP packet with the specified payload size.

Understanding Output

Example:

```
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.345 ms
```

`64 bytes`

Number of bytes received in the response.

`icmp_seq=1`

Sequence number of the ICMP packet.

`ttl=64`

Time To Live value in the received IP packet.

TTL limits how many routing hops a packet can survive.

`time=0.345 ms`

Round-trip time between the source and destination.

SUNBEAM

Summary Output

Example:

```
4 packets transmitted, 4 received, 0% packet loss
round-trip min/avg/max/mdev = 0.345/0.420/0.510/0.065 ms
```

Important Values

- `packets transmitted`
  - Number of packets sent.
- `packets received`
  - Number of replies received.
- `packet loss`
  - Percentage of packets for which no reply was received.
- `min`
  - Minimum round-trip time.
- `avg`
  - Average round-trip time.
- `max`
  - Maximum round-trip time.

SUNBEAM

3. traceroute

Description

- traceroute is used to identify the network path taken toward a destination.
- It displays intermediate routers between the source and destination.
- Each intermediate router is called a **hop**.
- It is useful for troubleshooting routing and connectivity problems.
- Traditional IPv4 traceroute commonly uses probes with increasing TTL values.
- The exact probe protocol can vary depending on the implementation and options.

Syntax

```
traceroute [OPTIONS] destination
```

Example

```
traceroute 8.8.8.8
```

or:

```
traceroute google.com
```

SUNBEAM

Important Options

-n

Disables DNS name resolution.

```
traceroute -n 8.8.8.8
```

Instead of displaying hostnames, it displays IP addresses.

This can make traceroute faster because reverse DNS lookups are avoided.

`-m`

Specifies the maximum number of hops.

```
traceroute -m 20 google.com
```

The traceroute stops after a maximum of 20 hops.

`-w`

Specifies the timeout for each probe.

```
traceroute -w 2 google.com
```

Waits approximately 2 seconds for a response.

`-q`

Specifies the number of probes sent for each hop.

SUNBEAM

```
traceroute -q 3 google.com
```

Sends 3 probes for each hop.

Understanding Output

Example:

```
1  192.168.1.1     1.123 ms  1.010 ms  1.100 ms
2  10.10.0.1       5.234 ms  5.100 ms  5.200 ms
3  172.16.0.1     10.321 ms 10.200 ms 10.300 ms
```

`1`, `2`, `3`

Hop number.

`192.168.1.1`

IP address of the router responding at that hop.

`1.123 ms`

Round-trip time for one probe.

Meaning of `* * *`

Example:

SUNBEAM

```
5  * * *
```

This means that probes at that hop did not receive a response within the timeout.

It does not automatically mean that the router or network is broken.

Possible reasons include:

- Router does not respond to traceroute probes.
- Firewall filtering.
- ICMP/UDP filtering.
- Rate limiting.
- Network security policies.

---

4. `ss`

Description

- `ss` stands for **Socket Statistics**.
- It is used to display information about network sockets.
- It can display:
  - TCP connections
  - UDP connections
  - Listening ports
  - Established connections
  - Socket states
  - Process information
- It is commonly used instead of the older `netstat` command.
- It is especially useful for troubleshooting services such as:
  - SSH

SUNBEAM

- HTTP
- HTTPS
- DNS
- Database services

Syntax

```
ss [OPTIONS]
```

Basic Command

```
ss
```

Displays socket information.

Show Listening TCP Ports

```
ss -ltn
```

Options

- `-l`
  - Show listening sockets.
- `-t`

SUNBEAM

    - Show TCP sockets.

- `-n`
    - Show numeric addresses and port numbers.
    - Avoids DNS/service-name resolution.

Example Output

```
State   Local Address:Port
LISTEN  0.0.0.0:22
LISTEN  0.0.0.0:80
```

This means:

- TCP port `22` is listening.
- TCP port `80` is listening.

Show Listening UDP Ports

```
ss -lun
```

Options

- `-l`
    - Listening sockets.
- `-u`
    - UDP sockets.

SUNBEAM

- `-n`
    - Numeric output.

Show All TCP Connections

```
ss -tan
```

Options

- `-t`
    - TCP.
- `-a`
    - All sockets.
- `-n`
    - Numeric output.

Show Process Information

```
sudo ss -tulpn
```

Options

- `-t`

SUNBEAM

    - TCP.
- `-u`
    - UDP.
- `-l`
    - Listening.
- `-p`
    - Show process information.
- `-n`
    - Numeric output.

Example

```
LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=1234,fd=3))
```

This tells us:

- Port `22` is listening.
- The service is `sshd`.
- Process ID is `1234`.

Important TCP States

`LISTEN`

The server is waiting for incoming connections.

SUNBEAM

`ESTAB`

An established TCP connection exists.

`TIME-WAIT`

The connection has been closed, but the socket is temporarily retained to handle delayed packets correctly.

`CLOSE-WAIT`

The remote side has closed its connection, but the local application has not completely closed its socket.

Check SSH Port

```
sudo ss -ltnp | grep ':22'
```

If SSH is listening on the standard port, output may contain:

```
LISTEN 0 128 0.0.0.0:22
```

---

5. `netstat`

Description

- `netstat` stands for **Network Statistics**.
- It displays information about:
  - Network connections
  - Listening ports

SUNBEAM

    - Routing tables
    - Network interfaces
    - Network statistics
- It is a traditional Linux networking command.
- On many modern Linux distributions, `netstat` is not installed by default.
- The `ss` command is generally preferred on modern Linux systems.

Syntax

```
netstat [OPTIONS]
```

Show Listening TCP Ports

```
netstat -lnt
```

Options

- `-l`
    - Listening sockets.
- `-n`
    - Numeric addresses and ports.
- `-t`
    - TCP.

Example Output

SUNBEAM

```
Proto Local Address    State
tcp   0.0.0.0:22       LISTEN
tcp   0.0.0.0:80       LISTEN
```

This means:

- TCP port `22` is listening.
- TCP port `80` is listening.

Show Listening TCP/UDP Ports

```
sudo netstat -lntup
```

Options

- `-l`
    - Listening.
- `-n`
    - Numeric output.
- `-t`
    - TCP.
- `-u`
    - UDP.

SUNBEAM

- `-p`
    - Show process/program information.

Show Routing Table

```
netstat -r
```

or:

```
netstat -rn
```

Example

```
Destination     Gateway         Genmask
0.0.0.0         192.168.1.1     0.0.0.0
192.168.1.0     0.0.0.0         255.255.255.0
```

Important Information

- `0.0.0.0`
    - Represents the default route when used as the destination.
- `192.168.1.1`
    - Gateway.
- `192.168.1.0`

SUNBEAM

    - Network destination.
- `255.255.255.0`
    - Network mask.

Show Network Interface Statistics

```
netstat -i
```

Displays information such as:

- Interface
- MTU
- RX packets
- TX packets
- RX errors
- TX errors
- Dropped packets

`netstat` vs `ss`

`netstat`	`ss`
Older/legacy tool	Modern tool
May not be installed by default	Usually available
Used for network statistics	Primarily used for socket statistics
Can display routing information	Focused mainly on sockets

SUNBEAM

netstat	ss
Slower for large socket tables	Generally faster
Still useful on older systems	Preferred on modern Linux

6. dig

Description

- dig stands for **Domain Information Groper**.
- It is a DNS query and troubleshooting utility.
- It allows us to query DNS servers directly.
- It can retrieve different DNS record types.
- It is useful for detailed DNS troubleshooting.

Syntax

```
dig [OPTIONS] DOMAIN [RECORD_TYPE]
```

Example

```
dig google.com
```

SUNBEAM

Basic DNS Query

```
dig google.com
```

By default, this generally performs an A record query.

The response contains sections such as:

```
QUESTION SECTION
ANSWER SECTION
AUTHORITY SECTION
ADDITIONAL SECTION
```

Query Specific Record Types

A Record

```
dig google.com A
```

Returns IPv4 address information.

AAAA Record

```
dig google.com AAAA
```

Returns IPv6 address information.

SUNBEAM

MX Record

```
dig google.com MX
```

Returns mail server information.

NS Record

```
dig google.com NS
```

Returns name server information.

TXT Record

```
dig google.com TXT
```

Returns TXT records.

Short Output

```
dig google.com +short
```

Example:

SUNBEAM

```
142.250.x.x
```

`+short` displays a simplified result instead of the complete DNS response.

Query a Specific DNS Server

```
dig @8.8.8.8 google.com
```

Here:

```
@8.8.8.8
```

specifies the DNS server that should be queried.

Another example:

```
dig @1.1.1.1 google.com
```

Understanding DNS Output

Example:

```
;; ANSWER SECTION:
```

SUNBEAM

```
google.com.  300  IN  A  142.250.x.x
```

`google.com.`

Domain name.

`300`

TTL — Time To Live.

It indicates how long the DNS response may generally be cached.

`IN`

Internet class.

`A`

DNS record type.

`142.250.x.x`

IPv4 address returned by DNS.

---

7. `nslookup`

Description

- `nslookup` stands for **Name Server Lookup**.
- It is used to query DNS servers.
- It can perform forward DNS lookups:
    - Domain name → IP address

SUNBEAM

- It can also perform reverse lookups:
  - IP address → Domain name
- It is available on Linux and Windows.
- It is useful for basic DNS troubleshooting.

Syntax

```
nslookup [OPTIONS] DOMAIN
```

Basic DNS Lookup

```
nslookup google.com
```

Example output:

```
Server:         192.168.1.1
Address:        192.168.1.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.x.x
```

Important Output

`Server`

DNS server that answered the query.

SUNBEAM

`Address`

IP address of the DNS server or the resolved domain, depending on where it appears in the output.

`Non-authoritative answer`

The response came from a recursive/caching DNS server rather than directly from an authoritative DNS server for that domain.

Reverse DNS Lookup

```
nslookup 8.8.8.8
```

This attempts to find a hostname associated with the IP address.

Query a Specific DNS Server

```
nslookup google.com 8.8.8.8
```

Here:

- `google.com`
  - Domain being queried.
- `8.8.8.8`
  - DNS server used for the query.

Query Specific Record Type

SUNBEAM

Start interactive mode:

```
nslookup
```

Then:

```
> set type=MX
> google.com
```

For NS records:

```
> set type=NS
> google.com
```

For A records:

```
> set type=A
> google.com
```

SUNBEAM

---

dig vs nslookup

Feature	dig	nslookup
DNS lookup	Yes	Yes

Feature	dig	nslookup
A record	Yes	Yes
AAAA record	Yes	Yes
MX record	Yes	Yes
NS record	Yes	Yes
Detailed DNS information	Excellent	Moderate
Short output	`+short`	Less convenient
DNS troubleshooting	Very useful	Useful
Available on Windows	Usually not by default	Yes
Linux usage	Common	Common

Quick Comparison of Commands

Command	Main Purpose
`ip`	Inspect/configure interfaces, IP addresses, routes and neighbors
`ping`	Test host reachability and measure round-trip time
`traceroute`	Show the network path toward a destination
`ss`	Inspect sockets, connections and listening ports
`netstat`	Inspect connections, ports, routing and network statistics
`dig`	Perform detailed DNS queries
`nslookup`	Perform basic DNS lookups

SUNBEAM

Common Network Troubleshooting Commands

Check IP Address

```
ip addr
```

Check Network Interfaces

```
ip link
```

Check Routing Table

```
ip route
```

Check Default Gateway

```
ip route | grep default
```

Check Neighbor/ARP Information

```
ip neigh
```

SUNBEAM

Test Connectivity

```
ping 192.168.1.1
```

Send Limited Number of Ping Requests

```
ping -c 4 192.168.1.1
```

Find Network Path

```
traceroute 8.8.8.8
```

Show Listening TCP Ports

```
ss -ltn
```

Show Listening TCP/UDP Ports

```
sudo ss -lntup
```

Check SSH Port

SUNBEAM

```
sudo ss -ltnp | grep ':22'
```

Check HTTP Port

```
sudo ss -ltnp | grep ':80'
```

Check HTTPS Port

```
sudo ss -ltnp | grep ':443'
```

Display Network Statistics

```
netstat -i
```

Check DNS Resolution

```
dig google.com
```

Get Only DNS Answer

```
dig google.com +short
```

SUNBEAM

Use a Specific DNS Server

```
dig @8.8.8.8 google.com
```

Basic DNS Lookup

```
nslookup google.com
```

Reverse DNS Lookup

```
nslookup 8.8.8.8
```

Example Network Troubleshooting Flow

Suppose a Linux machine cannot connect to:

```
192.168.1.100
```

A useful troubleshooting sequence is:

Step 1: Check Local IP

SUNBEAM

```
ip addr
```

Check:

- Correct network interface
- Correct IPv4 address
- Interface state

Step 2: Check Interface Status

```
ip link
```

Check whether the required interface is `UP`.

Step 3: Check Routing Table

```
ip route
```

Check:

- Destination network
- Default gateway
- Network interface

Step 4: Check Gateway Connectivity

SUNBEAM

```
ping 192.168.1.1
```

If the gateway cannot be reached, investigate the local network configuration first.

Step 5: Check Destination Connectivity

```
ping 192.168.1.100
```

If the destination does not respond, possible reasons include:

- Destination host is down.
- Destination is unreachable.
- ICMP is blocked.
- Host is on a different network.
- Firewall is filtering the traffic.

Step 6: Check Network Path

```
traceroute 192.168.1.100
```

This can help identify where packets stop receiving responses.

Step 7: Check Listening Services

On the destination Linux machine:

SUNBEAM

```
ss -ltn
```

Step 8: Check a Specific Service

For SSH:

```
sudo ss -ltnp | grep ':22'
```

For HTTP:

```
sudo ss -ltnp | grep ':80'
```

For HTTPS:

```
sudo ss -ltnp | grep ':443'
```

Step 9: Check DNS

If a hostname is being used instead of an IP address:

```
dig hostname.example.com
```

or:

SUNBEAM

```
nslookup hostname.example.com
```

Important Points to Remember

ip

Think:

> **What is my network configuration?**

```
ip addr
ip link
ip route
ip neigh
```

ping

Think:

> **Can I reach the destination?**

```
ping 192.168.1.1
```

traceroute

SUNBEAM

Think:

> **What path does the packet take?**

```
traceroute 8.8.8.8
```

---

ss

Think:

> **Which ports and connections are active?**

```
ss -ltn
```

---

netstat

Think:

> **Traditional tool for network statistics and connections.**

```
netstat -lnt
```

---

dig

Think:

SUNBEAM

> **What information does DNS provide?**

```
dig google.com
```

---

nslookup

Think:

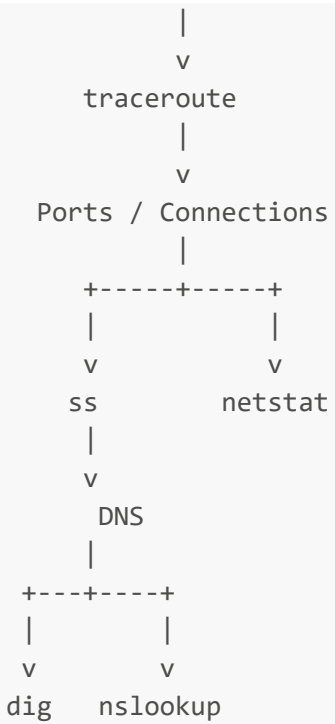
> **Can I perform a basic DNS lookup?**

```
nslookup google.com
```

Easy Way to Remember

```
Network Configuration
        |
        v
       ip
        |
        v
  Connectivity
        |
        v
      ping
        |
        v
  Network Path
```

SUNBEAM



Command Summary

```
ip          -> IP addresses, interfaces, routes, neighbors
ping        -> Reachability and round-trip time
traceroute  -> Network path / hops
ss          -> Sockets, ports and connections
netstat     -> Network statistics and connections
dig         -> Detailed DNS queries
nslookup    -> Basic DNS queries
```

SUNBEAM

Linux Root File System Structure

In Linux, the file system follows a **hierarchical structure** starting from the **root directory (/)**. Below is an overview of the **main directories** under the root (/) and their purposes.

---

1. Root Directory (/)

- The top-level directory from which all other directories branch.
- It contains essential system files and directories.

---

2. Important Directories Under Root (/)

Directory	Description
/bin	Essential **binary executables** (e.g., `ls`, `cp`, `mv`, `cat`).
/sbin	System binaries (e.g., `fsck`, `reboot`, `iptables`) used by **root/admin**.
/boot	Boot-related files (e.g., Linux **kernel (`vmlinuz`)**, bootloader (`grub`)).
/dev	Device files for hardware (`/dev/sda`, `/dev/tty`, `/dev/null`).
/etc	System-wide configuration files (`/etc/passwd`, `/etc/hosts`, `/etc/fstab`).
/home	User home directories (`/home/user1`, `/home/user2`).
/lib	Shared libraries needed for basic system binaries.
/lib64	64-bit libraries (on 64-bit systems).
/media	Mount point for **removable media** (`/media/usb`, `/media/cdrom`).
/mnt	Temporary mount point for file systems (`/mnt/data`).
/opt	Optional software packages (`/opt/google/chrome`).
/proc	Virtual directory for **process information** (`/proc/1`, `/proc/cpuinfo`).
/root	**Root user's home directory** (not the same as /).
/run	Runtime variable data (like process IDs: `/run/sshd.pid`).
/srv	Data for **services** (like web servers: `/srv/www`).
/sys	Virtual filesystem for system information (`/sys/devices`).
/tmp	Temporary files (`/tmp/session.log`), cleared on reboot.
/usr	User binaries, libraries, documentation (`/usr/bin`, `/usr/lib`).
/var	Variable data: logs, cache, spool (`/var/log`, `/var/tmp`).

3. Important Subdirectories

`/usr` (User Utilities and Applications)

- `/usr/bin/` → User binaries (`gcc`, `python`, `git`).
- `/usr/sbin/` → System administration binaries.

SUNBEAM

- `/usr/lib/` → Libraries for `/usr/bin/` and `/usr/sbin/`.
- `/usr/local/` → Locally installed software.

`/var` (Variable Data)

- `/var/log/` → Log files (`syslog`, `dmesg`, `auth.log`).
- `/var/spool/` → Print jobs, mail queues.
- `/var/cache/` → Cached data.

`/proc` (Process Information)

- `/proc/cpuinfo` → CPU details.
- `/proc/meminfo` → Memory usage.
- `/proc/[PID]/` → Running process details.

`/dev` (Device Files)

- `/dev/sda` → Primary hard drive.
- `/dev/null` → Discard output.
- `/dev/tty` → Terminals.

SUNBEAM

Storage Management (Basic + LVM)

◇ Section 1: Basic Disk Partitioning and Mounting

```
# list the block devices
lsblk

# list all partitions
sudo fdisk -l

# manage partitions on disk
sudo fdisk /dev/sda

# create filesystem (ext4)
sudo mkfs.ext4 /dev/sda1

# create filesystem (xfs)
sudo mkfs.xfs /dev/sda2

# create mount points
sudo mkdir /mnt/dir1
sudo mkdir /mnt/dir2

# mount partitions
sudo mount /dev/sda1 /mnt/dir1
sudo mount /dev/sda2 /mnt/dir2

# check disk usage
df -h

# edit fstab for permanent mount
sudo vim /etc/fstab

# add following entries
# partition     mount point     fs      defaults        defaults
/dev/sda1       /mnt/dir1       ext4    defaults        0 0
/dev/sda2       /mnt/dir2       ext4    defaults        0 0
```

SUNBEAM

◇ Section 2: LVM Creation and Mounting

```
# list block devices
lsblk

# list partitions
sudo fdisk -l
```

```
# create partitions on disks
sudo fdisk /dev/sda
sudo fdisk /dev/sdb

# create physical volumes
sudo pvcreate /dev/sda1
sudo pvcreate /dev/sdb1

# display physical volumes
sudo pvdisplay

# create volume group
sudo vgcreate myvg /dev/sda1 /dev/sdb1

# display volume group
sudo vgdisplay

# create logical volumes
sudo lvcreate -n lv1 -L 1024M myvg
sudo lvcreate -n lv2 -L 2.5G myvg

# display logical volumes
sudo lvdisplay

# create filesystem on logical volumes
sudo mkfs -t ext4 /dev/myvg/lv1
sudo mkfs -t ext4 /dev/myvg/lv2

# mount logical volumes
sudo mount /dev/myvg/lv1 /mnt/dir1
sudo mount /dev/myvg/lv2 /mnt/dir2

# check mounted filesystems with type
df -Th

# edit fstab for permanent mount
sudo vim /etc/fstab

# add following entries
# partition          mount point     fs      defaults        defaults
/dev/mapper/myvg-lv1 /mnt/dir1      ext4    defaults         0 0
/dev/mapper/myvg-lv2 /mnt/dir2      ext4    defaults        0 0
```

SUNBEAM

◇ Section 3: Extending LVM

```
# list block devices
lsblk

# create partition on new disk
sudo fdisk /dev/sdc
```

```
# create physical volume
sudo pvcreate /dev/sdc1

# extend volume group
sudo vgextend myvg /dev/sdc1

# extend logical volume
sudo lvextend /dev/myvg/lv2 -L 4.5G

# resize filesystem (ext4)
sudo resize2fs /dev/myvg/lv2 4608M

# check updated disk usage
df -h
```

systemd services

```
# get the list of unit files managed by systemd
> sudo systemctl list-unit-files

# get the list of unit files of a selected type
# > sudo systemctl list-units --type=<type>
> sudo systemctl list-units --type=service
> sudo systemctl list-units --type=mount
```

SUNBEAM